

Exam Preparation App with Adaptive Quizzes

A Project Report Submitted in Partial Fulfillment of the Requirements for the Award
of the Degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Konda Vaishali - 2520030117

P. Bindhu Madhavi - 2520030077

Alaparthi Nava Harsha - 2520030493

Section 7 | Team 3

Under the Esteemed Guidance of

Dr. Spandana. S

Department of Computer Science and Engineering



K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in



DECLARATION

The Project Report entitled “Exam Preparation App with Adaptive Quizzes” is a record of Bonafide work of KONDA VAISHALI - 2520030117, P. BINDHU MADHAVI – 2520030077, ALAPARTHI NAVA HARSHA - 2520030493 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Konda Vaishali - 2520030117

P. Bindhu Madhavi - 2520030077

Alaparathi Nava Harsha - 2520030493

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “Exam Preparation App with Adaptive Quizzes”, submitted by Konda Vaishali (2520030117), P. Bindhu Madhavi (2520030077), and Alaparathi Nava Harsha (2520030493), is submitted for the course Database Systems Engineering and Distributed Backend Development (25CS1302E), during the academic year 2026-2027, Trimester 4.

The report has been prepared under the guidance of Dr. Spandana. S, Department of Computer Science and Engineering. Approval of the work and its acceptance for academic submission are to be confirmed by the authorised faculty signatures below.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely help, support, and guidance provided by many people. We would like to express our sincere gratitude to everyone who assisted us directly or indirectly in completing our project, "Exam Preparation App with Adaptive Quizzes."

We express our deepest appreciation to our Course Coordinator and Project Guide, Dr. Spandana S, Department of Computer Science and Engineering, for her valuable guidance, continuous support, encouragement, and constructive suggestions throughout the development of this project.

We express our gratitude to the Head of the Department, Department of Computer Science and Engineering, for providing us with the necessary facilities, resources, and support required to successfully complete this project-based work.

We also thank all the teaching and non-teaching staff members of the Department of Computer Science and Engineering for their valuable assistance and cooperation during the course of this project.

Finally, we sincerely thank our parents, friends, and classmates for their constant encouragement, support, and cooperation throughout the completion of this project.

Konda Vaishali - 2520030117

P. Bindhu Madhavi - 2520030077

Alaparathi Nava Harsha - 2520030493

ABSTRACT

The Exam Preparation App with Adaptive Quizzes is a project for personalized exam practice and learner progress management. Its proposed Android version uses Item Response Theory (IRT) to select questions according to a learner's estimated ability. The project addresses the need for structured practice, clear feedback, and reliable management of questions, attempts, and results.

The proposed architecture separates presentation, validation, application logic, data access, and persistence. A normalized relational design organizes learners, subjects, topics, questions, answer options, quiz attempts, and responses. SQL queries support retrieval, joins, aggregation, and performance analysis. Local transactions are intended to keep related updates consistent, while indexes support frequent question and result lookups.

The supplied implementation is a self-contained HTML, CSS, and JavaScript prototype. It provides subject selection, randomized multiple-choice quizzes, answer explanations, score calculation, grading, learning links, and a browser-local leaderboard. It also requires camera and microphone recording before a quiz can begin. Results are stored in browser localStorage.

The Android design in the project abstract and review presentation specifies Kotlin, Room Database, and Firebase. IRT-based adaptation, persistent topic-level analytics, weak-topic detection, mock-test scheduling, and multi-device synchronization remain part of that proposed design; they are not implemented in the supplied HTML. This report documents the working browser logic and develops a database design for the intended Android system without treating the proposed components as completed features. Keywords: adaptive quizzes, IRT, relational database, SQL, Room, Firebase, performance analysis, exam preparation.

INDEX

S. No.	Chapters	Topics	Page
1	Introduction	1.1 Background 1.2 Problem statement 1.3 Scope 1.4 Objectives 1.5 Importance 1.6 Target users	7–21
2	System Requirements	2.1 Hardware 2.2 Software 2.3 Development tools	22–28
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version control 3.5 External services	29–40
4	System Architecture	4.1 Architecture diagrams 4.2 Layers 4.3 Deployment architecture	41–48
5	Design	5.1 Interface and workflow 5.2 ER diagram and schema 5.3 Normalization and integrity	49–61
6	Implementation	6.1 Modules 6.2 Front-end logic 6.3 API integration 6.4 Authentication and authorization	62–63
7	Features	7.1 Main features 7.2 Feature behaviour	64–66
8	Testing	8.1 Front-end and backend testing 8.2 Integration testing 8.3 Tools 8.4 Test cases and results	67–72
9	Deployment	9.1 Deployment steps 9.2 Configuration 9.3 Frontend and backend hosting	73–76
10	Challenges and Limitations	10.1 Issues 10.2 Solutions 10.3 Known limitations	77–79
11	Future Enhancements	11.1 Planned features 11.2 Integrations and optimization	80–82
12	Conclusion	12.1 Summary 12.2 Achievements 12.3 Learning outcomes	83–85
13	References	Project sources and technical documentation	86
14	Appendices	A. Application screenshots B. Code and SQL examples C. Installation and user guide D. Guide photographs E. Review and signature record	87–95

CHAPTER 1 INTRODUCTION

1.1 Background of the Project

Exam preparation is an important part of a student's academic journey and involves continuous practice, revision, self-assessment, and improvement. Students generally use textbooks, classroom notes, question banks, previous examination papers, and online learning resources to prepare for examinations. However, these methods may not always provide a personalized learning experience. A student may have a strong understanding of one topic while struggling with another topic. Therefore, providing the same fixed set of questions to every learner may not be equally effective.

An effective exam preparation system should provide students with an organized platform where they can practice questions, receive immediate feedback, review their scores, and identify areas that require additional attention. The Exam Preparation App with Adaptive Quizzes is proposed to address these requirements by combining an interactive examination interface with structured database management and adaptive quiz functionality.

The application provides a practice cycle in which a student can select a subject, choose the required number of questions, answer the questions, receive feedback, and review the final score. The system organizes questions based on subjects, topics, and difficulty levels so that students can practice different areas of their syllabus in a structured manner.

One of the important features of the proposed system is adaptive quiz functionality. In traditional examination system, all students may receive questions without considering their previous performance. In an adaptive system, the student's previous responses and performance can be considered while selecting questions of appropriate difficulty. This approach helps provide a more personalized practice experience and allows students to focus on areas where they need improvement.

The application also maintains student learning records. Instead of treating every quiz as an isolated activity, the system stores information about students, questions, quiz attempts, responses, scores, and performance. This makes it possible to distinguish between different attempts and analyze a student's progress over time.

From a database engineering perspective, every response is associated with a particular learner, question, and quiz attempt. Maintaining these relationships is important for generating accurate results and avoiding confusion between multiple attempts. Stable identifiers are used to distinguish students, questions, quizzes, and attempts. Reusable question information is maintained separately from attempt-specific information, which helps reduce unnecessary duplication of data.

The system also supports the management of subjects and questions. Questions can be organized according to their subject, topic, and difficulty level. This organization makes question retrieval more efficient and allows the quiz system to select questions according to the requirements of a particular practice session.

Another important aspect of the project is performance analysis. After completing a quiz, the student can view the score and understand their performance. The stored results can be used to identify topics in which the student performs well and topics that require further practice. This provides students with useful information for planning their future revision.

The project also demonstrates the integration of front-end, backend, and database components. The front-end provides the interface through which students interact with the application. The backend manages application logic, quiz processing, question selection, and result generation. The database provides persistent storage for student information, questions, quiz attempts, responses, and results.

The proposed application can be useful for students preparing for academic examinations because it provides a centralized environment for practice and assessment. Instead of using separate resources for questions, practice tests, and performance tracking, students can access these activities through a single platform.

From a software engineering perspective, the application follows a modular approach in which different functionalities such as question management, quiz processing, result calculation, and data storage can be developed and maintained separately. This improves the maintainability of the application and provides scope for adding additional features in the future.

The project also provides scope for further enhancement. Future versions can include more advanced adaptive learning techniques, detailed performance analytics, topic-wise recommendations, learning schedules, peer rankings, multimedia learning resources, and additional subjects. These features can further improve the usefulness of the platform for students.

The project is therefore based on three major observations:

1. Students require continuous and organized practice to improve their examination performance.
2. Different students have different levels of understanding, making personalized and adaptive practice more useful than a completely fixed question set.
3. Student performance data should be stored and analyzed systematically so that learners can understand their progress and identify weak areas.

By addressing these requirements, the Exam Preparation App with Adaptive Quizzes provides an integrated platform for digital exam preparation. It combines interactive quizzes, adaptive question selection, database management, result generation, and performance tracking to support a more organized and personalized learning experience.

1.2 Problem Statement

Problem Statement

The main problem addressed by this project is the development of an Exam Preparation App with Adaptive Quizzes that can provide suitable practice questions, deliver clear feedback, and maintain consistent learner progress. Students have different levels of understanding and may perform differently across subjects and topics. A conventional fixed-question examination system may not effectively address these differences.

The proposed system should support the organized management of questions based on subjects, topics, and difficulty levels. It should allow students to select a subject, attempt quizzes, submit answers, receive feedback, and view their results. The system should also maintain the relationship between students, questions, quiz attempts, responses, and results so that previous learning activities can be stored and analyzed accurately.

Another important requirement is the ability to track learner progress across multiple quiz attempts. Students should be able to distinguish their current performance from previous attempts and identify topics where additional practice is required. Maintaining persistent records is therefore essential for meaningful performance analysis and personalized learning.

The supplied browser prototype demonstrates the basic examination interface and practice workflow. However, it has important data and learning limitations. The prototype stores summary results in a local JSON array and does not maintain complete response history for individual questions. It also does not estimate learner ability or use previous performance to determine suitable question difficulty.

These limitations create a gap between a basic examination demonstration and a complete adaptive exam preparation application. The proposed system aims to overcome this gap by introducing structured database storage, persistent quiz and response records, adaptive question selection, result analysis, and learner performance tracking.

Therefore, the problem is to design and develop an integrated exam preparation application that combines interactive quiz functionality, database management, adaptive question selection, persistent learning records, and performance analysis to provide students with a more organized and personalized examination preparation experience.

1.3 Scope of the Project

Scope and Project Coverage

The scope of this project is to design and develop an Exam Preparation App with Adaptive Quizzes that provides students with a structured and personalized environment for examination preparation. The application is intended to combine quiz practice, question management, learner performance tracking, database management, and adaptive question selection within a single platform. The primary objective is to reduce the limitations of conventional fixed-question practice systems and provide students with a more organized method of preparing for examinations.

The project covers the development of an examination interface through which students can interact with subjects, topics, questions, quizzes, and results. A student can select a subject and begin a practice session based on the available questions. The student can answer questions one by one and complete the quiz within the application. After submission, the system can calculate the result and provide feedback regarding the student's performance.

The proposed system includes student-related information as one of its major components. Each student can be identified using a unique identifier so that quiz attempts and performance records can be associated with the correct learner. Student information can be maintained separately from quiz and question information to avoid unnecessary duplication. The student management component provides the foundation for maintaining learner-specific records. It also makes it possible to distinguish the performance of different students and maintain multiple quiz attempts for the same student.

The application is designed to organize examination preparation according to subjects and topics. Questions can be associated with a particular subject and topic so that students can practice specific areas of their syllabus. For example, subjects such as Java, Python, C, Operating Systems, and Machine Learning can contain multiple topics and questions. This organization allows the application to retrieve relevant questions and provides students with a structured way to prepare for different subjects.

The question bank is an important component of the proposed application. It contains reusable question information that can be used across multiple quiz attempts. Each question can have a unique identifier, subject, topic, difficulty level, question text, answer choices, and correct answer. The question bank separates reusable question content from individual learner attempts. This allows the same question to be used by multiple students or in multiple quiz sessions without creating duplicate question records. Questions can also be classified according to difficulty levels such as Easy, Medium, and Hard. This classification is useful for the adaptive quiz component because the system can use the difficulty information while selecting questions for a learner.

The quiz module manages individual practice sessions. A quiz can contain a selected number of questions based on the student's requirements. The system records when a quiz attempt begins and when it is completed. Each quiz attempt receives a unique identifier. This identifier allows the system to associate individual responses with a particular examination session. Therefore, two attempts made by the same student can be stored and analyzed separately. The quiz management component also supports result calculation. After the student submits the quiz, the system can compare the submitted answers with the correct answers and calculate the corresponding score.

A major scope of the project is maintaining individual learner responses. Instead of storing only the final score, the proposed database design records which student answered which question during a particular attempt. This relationship is important because a student may answer the same question differently in different attempts. Maintaining response-level information allows the system to preserve the complete history of the learner's practice. The response records can contain information such as the attempt identifier, question identifier, selected answer, correctness, and response information. These records can later be used for performance analysis and adaptive question selection.

The main distinguishing feature of the proposed application is adaptive quiz functionality. Traditional quiz systems generally provide a fixed set of questions to every learner. However, students may have different levels of knowledge and different learning requirements. The proposed adaptive approach considers learner performance while selecting suitable questions. For example, if a student consistently answers easy questions correctly, the system can provide questions of a higher difficulty level. If the student has difficulty answering questions from a particular topic, additional practice from that topic can be provided. The adaptive-selection workflow is therefore intended to make the practice experience more

personalized. The exact adaptive algorithm and ability-estimation mechanism are treated as design components of the proposed system and can be refined during implementation and testing.

The project also covers the analysis of student performance. After completing a quiz, the application can display the score and provide information about the student's performance. Stored quiz attempts can be used to calculate performance across multiple sessions. The system can identify topics where the student performs well and areas where additional practice may be required. Performance analysis can include information such as total questions attempted, correct answers, incorrect answers, score, percentage, and topic-wise performance. Such information can help students understand their strengths and weaknesses.

Database management forms an important part of the project. The proposed database design separates different types of information into logically related entities. These include students, subjects, questions, quizzes, quiz attempts, responses, and performance records. Each major entity can have a unique identifier, while relationships between entities can be maintained using appropriate keys. This structure helps maintain data consistency and allows the system to retrieve information efficiently. The proposed database design also reduces unnecessary duplication by storing reusable question information separately from attempt-specific response information.

The project scope includes SQL operations required for managing the application data. These operations include inserting new records, retrieving questions, updating information, deleting records where appropriate, and generating results from stored data. Queries can also be used to retrieve questions according to subject, topic, and difficulty level. Result-related queries can calculate scores and summarize learner performance across multiple attempts. SQL examples included in the report are intended to demonstrate how the proposed database structure can support the application's functional requirements.

The supplied browser prototype is included within the scope of the project as an initial demonstration of the examination workflow. The prototype demonstrates the basic interaction between the learner and the quiz interface. The prototype allows the learner to select a subject, choose the number of questions, answer questions, receive feedback, and view a score. It therefore provides a basic representation of the intended examination workflow. However, the prototype is primarily intended for interface demonstration and does not provide the complete persistent database and adaptive-learning functionality proposed for the final system.

The current browser prototype has several limitations that define the additional work required for the proposed application. It stores summary results in a local JSON structure rather than maintaining a complete relational database. The prototype also does not retain detailed response history for every question and attempt. As a result, it cannot provide comprehensive historical analysis of learner performance. Another limitation is that the prototype does not estimate learner ability or dynamically select questions based on previous performance. The proposed adaptive application addresses these limitations by introducing structured data storage, persistent learner records, and adaptive question-selection logic.

The proposed system includes an Android application architecture using Kotlin. Kotlin is intended to provide the application layer through which students interact with the examination and learning features. The proposed architecture separates the user interface, application logic, and data management components. This separation supports maintainability and makes it easier to modify individual components without affecting the complete application. The Android application is intended to provide a more complete implementation of the examination workflow demonstrated by the browser prototype.

The proposed Android architecture uses Room as the local database layer. Room provides a structured way of managing application data and is intended to store learner information, questions, quiz attempts, responses, and results locally. The use of a local database can allow the application to maintain learning records even when continuous network connectivity is not available. The locally stored information can later be synchronized with cloud storage according to the proposed synchronization policy.

The proposed architecture also includes Firebase as a cloud-based component. Firebase is intended to support centralized storage and synchronization of selected learner and application records. The synchronization policy described in this report is a design proposal that defines how local and cloud records can be maintained consistently. The purpose of synchronization is to reduce the possibility of losing learner progress and to support access to relevant records across supported devices. The exact synchronization mechanism, conflict resolution strategy, and deployment configuration may be refined during the implementation stage.

The application is designed with the objective of providing a simple and understandable user experience. Students should be able to navigate between subjects, quizzes, questions, and results without unnecessary complexity. The examination interface should clearly display the question, available options, selected answer, and navigation controls. After completing the quiz, the result should be

presented in a readable format so that the learner can easily understand the outcome.

The proposed system also considers basic data integrity requirements. Student records, question records, and quiz attempts should maintain consistent relationships. Unique identifiers and appropriate database constraints can help prevent invalid or duplicate relationships. The system should also ensure that a student's responses are associated with the correct quiz attempt. This is particularly important when a student completes multiple quizzes or repeats the same quiz.

The project provides a foundation for several future enhancements. The adaptive learning component can be improved using more advanced ability-estimation techniques and learning models. More detailed analytics can also be introduced to provide topic-wise progress charts and long-term performance trends. Future versions may include personalized study schedules, weak-topic recommendations, peer rankings, notifications, multimedia learning resources, video resources, voice-based learning features, and additional examination categories. The application can also be expanded to support a larger question bank and multiple educational institutions. Additional authentication and authorization mechanisms can be introduced to support different types of users, such as students, faculty members, and administrators.

Overall, the project covers the transition from a basic browser-based examination prototype to a proposed integrated adaptive exam preparation application. The system combines an interactive examination interface with structured database management and personalized quiz functionality. The report covers the supplied prototype and the proposed Android architecture using Kotlin, Room, and Firebase. The database schema, SQL examples, synchronization policy, and adaptive-selection workflow developed in this report are design proposals intended to make the proposed implementation concrete and reviewable.

The project does not claim that every proposed component is already fully implemented unless explicitly demonstrated through the application. Instead, the report distinguishes between the existing prototype and the proposed architecture so that the intended development process and technical design remain clear. The overall scope covers student and learner management, subject and topic organization, structured question-bank management, quiz creation and quiz attempts, individual response tracking, adaptive question selection, result generation and performance analysis, relational database design and SQL operations, Android application architecture using Kotlin, local data management using Room, proposed

cloud storage and synchronization using Firebase, and future support for advanced personalized learning features.

Through these components, the Exam Preparation App with Adaptive Quizzes aims to provide a structured, scalable, and personalized platform for students to practice examination questions, understand their performance, identify areas requiring improvement, and maintain their learning progress over multiple quiz attempts.

1.4 Objectives

The primary objective of the Exam Preparation App with Adaptive Quizzes is to develop an organized, user-friendly, and personalized platform that helps students prepare for examinations through continuous practice, revision, assessment, and performance analysis. The application is intended to provide a structured environment where learners can select subjects, attempt quizzes, receive feedback, review their results, and understand the areas in which they require additional preparation. The system aims to combine an interactive examination interface with reliable database management and adaptive learning concepts.

The application aims to provide a subject-based quiz interface through which students can select the required subject and practice questions according to their learning needs. Questions can be organized based on subjects, topics, and difficulty levels so that learners can access relevant practice material. The interface is intended to remain simple and understandable, allowing students to answer questions, navigate through a quiz, submit their responses, and view their results without unnecessary complexity.

Another important objective is to design a structured and normalized data model for maintaining learner information, subjects, questions, quizzes, attempts, responses, and performance records. The database should preserve the relationships between these entities so that the information stored by the application remains consistent and meaningful. A learner may attempt several quizzes, each quiz may contain multiple questions, and every response should be associated with the appropriate question and attempt. Maintaining these relationships is essential for reliable result generation and performance analysis.

The project also aims to separate reusable question information from attempt-specific data. Questions should be maintained in a question bank and reused across different learners and quiz attempts instead of being duplicated for every session. At the same time, individual responses and attempt information should be stored separately so that the system can maintain a complete history of the learner's

activities. This approach supports better data organization and reduces unnecessary duplication.

The application aims to support SQL-based retrieval and analysis of examination data. SQL operations can be used to retrieve questions based on subjects, topics, and difficulty levels and to obtain information about learner attempts and responses. Joins can be used to combine related information from different tables, while aggregation functions can support calculations such as scores, percentages, total attempts, correct answers, and performance summaries. Indexes can also be considered for frequently searched fields to improve the efficiency of question retrieval and progress analysis as the size of the database increases.

Another objective is to maintain consistency while storing quiz attempts and responses. When a learner completes a quiz, the attempt information and its associated responses should be stored correctly. Local transaction boundaries can be defined so that related database operations are treated consistently. This helps prevent situations in which an attempt is recorded without all its responses or where incomplete data is used to calculate a result.

A major objective of the project is to support adaptive quiz functionality. Unlike a fixed examination system in which every learner receives the same type of questions, the proposed application is intended to consider learner performance and question difficulty while selecting practice questions. The adaptive approach can provide questions that are more appropriate to the learner's current level and can encourage additional practice in areas where the learner experiences difficulty.

The project also proposes an IRT-based adaptive quiz engine as an advanced component. Item Response Theory can be used to relate question difficulty with learner ability and support more meaningful question selection. However, reliable IRT-based adaptation requires suitable question-bank difficulty information and sufficient calibration and response data. Therefore, this component is considered part of the proposed adaptive architecture and can be implemented and refined as the required data becomes available.

Another objective is to maintain learner performance records over multiple attempts. Instead of displaying only the result of the current quiz, the application should provide a foundation for storing previous attempts and comparing performance over time. Such records can help learners understand whether their preparation is improving and can provide useful information for future revision.

The system also aims to support identification of weak topics through response analysis. Questions can be associated with specific topics, allowing the application to

analyze repeated responses and identify areas where a learner may need additional practice. However, the system should avoid considering a topic weak based on only one incorrect answer. Meaningful analysis should be based on sufficient response history and repeated performance patterns.

The proposed application also aims to support persistent learning records using an Android-based architecture. The proposed design uses Kotlin for application development and Room for local database management. Firebase is considered for authenticated cloud records and synchronization. The purpose of this architecture is to provide a structured method for maintaining learner information and quiz data while allowing future expansion of the application.

Controlled synchronization between the local Android database and authenticated cloud records is another objective of the proposed system. Synchronization should be designed carefully so that learner records remain consistent and conflicts between local and cloud information can be handled appropriately. The system should avoid unnecessary duplication and should ensure that important learning records are not accidentally overwritten.

The project further aims to provide clear and understandable result summaries. After completing a quiz, students should be able to view their score and understand their performance. The result information can provide the basis for future extensions such as subject-wise analysis, topic-wise performance, historical progress, and personalized recommendations.

Finally, the project aims to provide a scalable foundation for future improvements in digital exam preparation. The modular nature of the proposed application can allow additional subjects, larger question banks, improved adaptive algorithms, detailed analytics, study schedules, notifications, multimedia resources, and other learning features to be introduced later. Overall, the objective is to develop an integrated examination preparation platform that combines quiz practice, database management, adaptive question selection, persistent learning records, and performance analysis to provide students with a more organized and personalized preparation experience.

1.5 Importance of the Application

The Exam Preparation App with Adaptive Quizzes is important because effective examination preparation requires continuous practice, revision, feedback, and understanding of mistakes. Students often use different sources such as textbooks, classroom notes, previous question papers, and online resources for preparation. Although these resources are useful, they may not provide a centralized mechanism

for recording practice activities and understanding individual performance. The proposed application provides an organized environment in which learners can practice questions, receive feedback, view results, and maintain their learning history.

One of the major benefits of the application is that it supports repeated practice. Students can attempt quizzes multiple times and use the results to understand their level of preparation. Repeated practice can help learners become more familiar with important concepts and question patterns. Instead of treating every quiz as an isolated activity, the proposed system provides a foundation for maintaining information about previous attempts and using that information for future learning.

Immediate feedback is also an important part of the application. When learners receive information about their answers, they can identify mistakes and connect the result with the corresponding concept. This can reduce the time between making an error and recognizing the need for revision. Clear result summaries can further help learners understand their overall performance after completing a quiz.

The application is particularly important because students do not have identical learning requirements. A learner may have a strong understanding of one subject or topic while experiencing difficulty in another. A fixed set of questions may therefore be too easy for one learner and too difficult for another. The proposed adaptive quiz approach provides a foundation for selecting questions according to learner performance and question difficulty, making practice more personalized.

The application is also important for identifying areas that require additional preparation. By maintaining responses and associating questions with subjects and topics, the system can analyze patterns in learner performance. If a student repeatedly makes mistakes in questions belonging to a particular topic, that topic can be considered for additional practice. Such analysis can help students focus their revision rather than spending equal amounts of time on every topic.

However, reliable topic-level analysis requires sufficient data. A single incorrect answer should not automatically classify a topic as weak because the mistake may have occurred due to several temporary factors. Maintaining a history of responses allows the system to make more meaningful observations based on repeated performance rather than isolated results.

The application is also important from a database engineering perspective. Examination systems involve relationships between students, questions, quizzes, attempts, and responses. A response cannot be interpreted correctly without knowing which learner submitted it, which question was answered, and which

attempt it belongs to. Therefore, proper database design is necessary to maintain these relationships and generate reliable results.

A well-structured database also helps prevent unnecessary duplication of information. Question content can be stored separately and reused across different attempts, while learner responses can be maintained as attempt-specific records. This separation improves organization and provides a stronger foundation for future analysis.

The application demonstrates the importance of SQL operations in a practical system. SQL queries can be used to retrieve questions, combine related records, calculate scores, analyze performance, and generate summaries. Joins and aggregation operations become particularly useful when information from multiple related tables must be combined. Indexes can also improve retrieval performance when frequently searched fields are properly identified.

Data consistency is another important aspect of the application. Displaying a score on the screen does not by itself prove that the result has been stored correctly. The application must also ensure that the corresponding attempt and responses are maintained consistently in the database. Transaction boundaries can therefore play an important role when multiple related records are being created or updated.

The application also highlights the importance of persistent learning records. If quiz results disappear after an application session ends, students cannot effectively compare their performance over time. Persistent storage allows previous attempts to remain available for future analysis. This can help learners understand whether their scores are improving and whether repeated practice is producing meaningful progress.

The proposed Android architecture further increases the potential usefulness of the application. Kotlin can be used for Android application development, while Room can provide local database management. Firebase can be considered for authenticated cloud records and synchronization. Together, these technologies provide a proposed foundation for extending the application beyond a simple browser-based demonstration.

The browser prototype is also important because it demonstrates the basic examination workflow and provides a starting point for further development. It shows how a learner can select a subject, choose the number of questions, answer questions, receive feedback, and view a result. At the same time, the limitations of the prototype demonstrate why structured database storage and persistent response history are required for a more complete adaptive application.

The proposed application is also valuable from an academic and engineering perspective. It brings together concepts from database management, software development, application architecture, SQL, transaction processing, adaptive learning, and data analysis. Instead of implementing these concepts independently, the project demonstrates how they can work together to solve a practical educational problem.

The system can also provide a foundation for future personalized learning features. As more learner response data becomes available, the adaptive selection mechanism can be improved. Future versions can provide personalized revision suggestions, study schedules, weak-topic recommendations, performance charts, notifications, and additional learning resources.

The application also has importance in terms of scalability. A properly structured database and modular application architecture can allow the question bank and number of learners to increase without requiring the complete system to be redesigned. Additional subjects and topics can be added while preserving the existing relationships between learners, questions, and attempts.

The proposed system also emphasizes the difference between local information and verified shared information. For example, a locally calculated leaderboard may display rankings based only on records available on one device. It cannot automatically represent a verified ranking across multiple learners and devices unless the relevant records are synchronized and validated through a trusted data source. This demonstrates why database design and synchronization policies are important when developing multi-user applications.

Overall, the importance of the Exam Preparation App with Adaptive Quizzes lies in its ability to combine learning, assessment, and data management within a single platform. The application provides students with a structured method for practicing questions and reviewing their performance while providing a technical foundation for adaptive learning and persistent records.

The project therefore goes beyond simply displaying examination questions. It demonstrates how learner responses can be stored, analyzed, and used to support future practice. Immediate feedback helps learners understand mistakes, stored history helps identify progress, and adaptive selection provides a pathway toward more personalized examination preparation.

By combining an interactive examination interface, structured database design, SQL operations, transaction consistency, persistent learner records, and proposed adaptive-learning functionality, the application provides a practical and extensible

solution for digital exam preparation. It also establishes a foundation that can be enhanced in future versions as more question calibration data and learner response history become available.

1.6 Target Users and Audience

User	Expected use
Learners	Practise questions, read explanations, and review personal progress.
Faculty and evaluators	Review the application workflow and its database engineering design.
Question-bank maintainers	Proposed role for adding, reviewing, and classifying questions.
Project team	Implement and test Android persistence, adaptation, and synchronization.

CHAPTER 2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware and Software Requirements

The Exam Preparation App with Adaptive Quizzes consists of a browser-based prototype and a proposed Android application. Therefore, the hardware and software requirements are considered separately for the prototype demonstration and the proposed mobile implementation. The requirements described here are intended to provide a practical environment for developing, testing, demonstrating, and extending the application without specifying numerical hardware benchmarks that have not been experimentally verified.

For demonstrating the supplied browser prototype, a desktop computer or laptop with a modern web browser is suitable. The system should be capable of running a current version of browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, or another browser that supports the web technologies used by the prototype. The browser should support HTML, CSS, JavaScript, local data handling, and the media features required by the application.

A stable display is required so that students can clearly view the examination interface, questions, answer options, navigation controls, feedback, and result summaries. A keyboard and mouse or touch-enabled input device can be used to interact with the prototype. The prototype should be demonstrated on a system capable of running the browser and application simultaneously without noticeable interruption.

If the prototype includes or is extended with video recording, voice recording, or other media-based features, the demonstration system should also provide a working camera and microphone. The browser must be able to request and receive permission to access these devices. Media permissions should be granted only when the corresponding functionality is being demonstrated. If a particular demonstration does not use camera or microphone functionality, these devices are not required for the basic quiz workflow.

The computer used for development should also support a suitable code editor or integrated development environment. Visual Studio Code can be used for editing the browser prototype, Java source files, database scripts, documentation, and other project resources. A suitable Java Development Kit can be installed when Java-based backend components are required. The required version depends on the selected backend framework and implementation environment.

The browser prototype does not prescribe a specific minimum RAM, processor speed, graphics capability, or storage capacity because no formal hardware benchmark has been measured for the prototype. Instead, the requirement is that the computer should be capable of running the selected browser, development tools, database tools, and project files together in a stable manner.

For the proposed Android version, an Android smartphone, tablet, or compatible Android emulator is required. The Android device or emulator should support the Android SDK version selected during implementation. The application should be tested against the Android version and device configuration targeted by the development environment.

The Android application is proposed to use Kotlin as the primary application development language. Android Studio can be used as the development environment for creating, building, testing, and running the application. The Android SDK and the required build tools should be installed and configured according to the selected project configuration.

The proposed application also uses Room for local database management. The Android device or emulator therefore needs sufficient local storage for maintaining the application, question bank, learner information, quiz attempts, responses, and other locally stored learning records. The amount of required storage depends on the size of the question bank and the amount of historical attempt data maintained by the application.

Offline practice is an important consideration for the proposed architecture. When questions and relevant learner records are available locally, the student can continue practice without requiring continuous network access. Local database storage is therefore intended to provide the basic information required for offline quiz activities.

Network connectivity becomes necessary when the application communicates with cloud-based services. The proposed architecture considers Firebase for authenticated cloud records and synchronization. Internet access is therefore required when learner records need to be synchronized with cloud storage or when cloud-based information is retrieved.

Network connectivity may also be required for external learning resources such as educational videos. If the application provides links to online learning videos related to subjects or topics, the student requires an active internet connection to access those external resources. The quality of the learning-video experience will depend on the available network connection and the external service.

The proposed cloud architecture also requires appropriate authentication and network configuration. The application should communicate with the cloud service through supported and secure mechanisms. Authentication information should not be stored or transmitted in an unsafe manner. The exact Firebase configuration, authentication mechanism, and synchronization approach are implementation details that can be finalized during development.

The application may also require permissions depending on the features included in the final Android implementation. Basic quiz functionality may not require access to device hardware beyond normal application operation. If features such as camera-based recording, microphone-based voice recording, notifications, or other device services are implemented, the corresponding Android permissions must be configured appropriately.

The software environment should also provide the necessary database and development tools. MySQL or another selected relational database system can be used during database development and testing when a server-side relational database is included in the implementation. Database management tools can be used to create tables, execute SQL queries, test relationships, and verify stored records.

The proposed database environment should support the operations required by the application, including insertion, retrieval, updating, joining, aggregation, and indexing. It should also support appropriate transaction handling for operations involving quiz attempts and responses. The exact database version should be selected according to the backend and deployment environment used for the final implementation.

The question bank is another important software resource required by the application. The database or local storage should contain properly structured questions with information such as subject, topic, difficulty level, options, and correct answers. For the adaptive component, additional question difficulty and calibration information may be required. The quality and quantity of this data directly affect the usefulness of adaptive question selection.

For an IRT-based adaptive implementation, the system requires suitable question parameters and sufficient learner-response data. Therefore, the software environment alone is not sufficient to provide reliable adaptive ability estimation. Appropriate question calibration and response history must also be available. The adaptive engine should be treated as a proposed component until the required data has been collected and validated.

The development environment should also support testing of the complete application workflow. Developers should be able to test subject selection, question retrieval, answer submission, result calculation, database storage, repeated quiz attempts, and performance analysis. For the proposed Android implementation, testing should include both online and offline scenarios where applicable.

The system should also be tested for data consistency. For example, when a student completes a quiz, the application should correctly store the attempt and its associated responses. If synchronization is implemented, the local and cloud records should be checked to ensure that valid data is transferred correctly. These tests help verify that the application behaves correctly beyond the visual interface.

The following general hardware and software environment is therefore considered suitable for the project. For the browser prototype, a desktop or laptop with a modern browser, keyboard and mouse, and optional camera and microphone is sufficient for demonstration. For development, the system should also support the required code editor, runtime environment, database tools, and project files.

For the proposed Android application, an Android-compatible smartphone, tablet, or emulator with support for the selected Android SDK is required. The device should have adequate local storage for the application and question bank. Internet connectivity is required for cloud synchronization, authenticated cloud operations, and access to external learning videos, while local question and attempt records can support offline practice where implemented.

No untested numerical minimum for RAM, processor speed, storage, camera resolution, microphone quality, or network bandwidth is specified in this report. Such values would depend on the final implementation, database size, number of users, media features, and selected Android configuration. Instead, the requirements focus on providing a practical environment capable of running the application and its supporting development tools.

Overall, the hardware and software requirements are designed to support the transition from the supplied browser prototype to the proposed adaptive Android application. The prototype can be demonstrated using a standard desktop or laptop environment, while the proposed mobile application can be developed and tested using Android Studio, Kotlin, Room, and the selected cloud services. This environment provides the necessary foundation for implementing quiz practice, persistent learner records, database operations, adaptive question selection, performance analysis, and future synchronization features.

2.2 Software Requirements

Layer	Browser prototype	Proposed Android version
Interface	HTML and CSS	Kotlin with Compose or XML
Application logic	Vanilla JavaScript	Kotlin services and view models
Local persistence	Browser localStorage	Room over SQLite
Cloud services	None integrated	Firebase authentication and data services
Media	getUserMedia and MediaRecorder	Android APIs if recording is retained
Execution	Browser; localhost or HTTPS preferred	Android Studio build and device installation

2.3 Development Tools and Frameworks

The development of the Exam Preparation App with Adaptive Quizzes involves a combination of development tools, programming technologies, database frameworks, cloud services, testing tools, and version-control systems. These technologies are selected to support the development of the user interface, application logic, database operations, adaptive quiz functionality, testing, and future deployment of the application.

Android Studio is proposed as the primary integrated development environment for the Android version of the application. It provides facilities for creating Android projects, writing and organizing source code, managing project dependencies, building the application, debugging errors, and testing the application on an emulator or physical Android device. Android Studio can also be used to monitor application behaviour during execution and identify problems related to the interface, database operations, or application logic.

The Android SDK provides the platform libraries and development components required to build and execute the Android application. The SDK version will be selected according to the target Android devices and compatibility requirements of the project. It provides the necessary tools for compiling the application, managing Android platform features, and testing the application in an appropriate environment.

Kotlin is proposed as the main programming language for the Android implementation. Kotlin is used to represent application logic and can be used for features such as subject selection, question retrieval, quiz processing, answer validation, score calculation, and result generation. Kotlin also supports structured

and modular application development, making it suitable for separating different responsibilities of the proposed system.

Room is proposed as the local database framework for the Android application. Room provides an abstraction layer over SQLite and allows application data to be represented using entities, data-access objects, and database classes. In this project, Room can be used to store students, subjects, questions, quizzes, quiz attempts, responses, and performance information.

The use of Room is important because the application needs to maintain learner records and quiz information persistently. Local storage can also support offline practice when the required questions and learner records are already available on the device. SQL operations can be tested independently before they are incorporated into Room data-access objects.

Firebase is proposed as the cloud component of the application. It can be used to support authenticated cloud records and synchronization between local application data and cloud-based information. Firebase can provide a foundation for maintaining learner-related information when cloud access is required. The exact Firebase services and configuration will depend on the final implementation.

HTML, CSS, and JavaScript are used for the supplied browser prototype. HTML provides the structure of the examination interface, CSS controls the presentation and layout, and JavaScript manages interactive behaviour such as subject selection, question display, answer processing, feedback, and result calculation. The prototype provides an initial demonstration of the examination workflow before the proposed Android implementation is completed.

A modern web browser is required to execute and test the supplied prototype. Browser developer tools can be used to inspect the HTML structure, CSS behaviour, JavaScript console messages, browser storage, and media permissions. These tools are useful for identifying errors and verifying the behaviour of the browser-based prototype.

Git is intended to be used as the version-control system for the project. It can track changes made to source code and other project files throughout development. Version control allows developers to maintain previous versions, review changes, and recover earlier versions when required.

GitHub can be used as the remote repository for storing and managing the project source code. It can also support collaboration between team members by providing a centralized location for project files and development history. However, GitHub

repository history should only be documented as completed if an actual repository and commit history are available.

Postman is proposed as a tool for testing backend APIs if backend services are included in the final implementation. It can be used to send HTTP requests and inspect responses independently of the application interface. This can help verify operations such as retrieving questions, submitting quiz responses, obtaining results, and accessing learner information.

Database development and testing can be performed using an appropriate SQL environment. SQL queries can be tested independently in an isolated SQLite or relational database environment before being incorporated into the application. This allows table relationships, joins, aggregation, constraints, and indexes to be verified before they become part of the application database layer.

The proposed development environment therefore combines front-end technologies, Android development tools, database frameworks, cloud services, testing utilities, and version-control tools. Each technology has a specific role in the overall system, while the combination of these technologies provides a foundation for developing a structured, maintainable, and scalable exam preparation application.

The tools listed above represent both the current prototype environment and the intended implementation environment. The supplied files currently demonstrate the browser-based examination interface, while the Android Studio, Kotlin, Room, Firebase, GitHub, and Postman components represent the proposed development and testing architecture where they have not yet been implemented. This distinction ensures that the project documentation accurately represents the current state of development and the planned technical direction.

CHAPTER 3 TECHNOLOGY STACK

3.1 Front-end

The supplied application contains the front-end implementation using HTML, CSS, and JavaScript. These technologies work together to provide the user interface and interactive behaviour of the exam preparation system. The current prototype is designed as a browser-based application in which different screens are represented within the same HTML document. This approach allows the prototype to demonstrate the main examination workflow without requiring multiple HTML pages or a separate frontend framework.

The HTML structure defines the different sections of the application and provides the basic elements required for student interaction. These elements include the application title, navigation controls, subject selection areas, question displays, answer options, feedback sections, quiz controls, and result information. The structure is organized so that different parts of the examination workflow can be displayed when required.

Each screen in the supplied prototype is represented as a section with the class `page`. The application uses the `showPage` function to control which section is visible to the learner. When a particular screen is selected, the function removes the active state from the other screens and assigns the active class to the selected screen. This creates a multi-screen experience within a single HTML document and avoids the need for a complete page reload whenever the learner moves from one stage of the application to another.

The front-end provides the basic workflow required for exam preparation. The learner can enter the application, select a subject, choose the required number of questions, start a quiz, answer questions, receive feedback, and view the final result. The interface therefore represents the complete basic practice cycle from question selection to result review.

The user interface uses several visual components to organize information clearly. Cards are used to group related information, while subject tiles provide an interactive way of selecting subjects. Buttons are used for navigation and quiz actions, and the question area provides the space in which individual questions and answer choices are displayed. Feedback panels are used to communicate the result of an answer or quiz activity, while the results table presents the final performance information in a structured format.

CSS is used to control the appearance, spacing, alignment, typography, and layout of the interface. The styling provides a consistent visual structure across the different

screens of the prototype. The use of cards, buttons, panels, and organized content areas makes the interface easier to understand and provides a clear separation between different activities.

The prototype also includes a basic responsive design adjustment. A CSS media query is used for screen widths below 700 pixels. This changes the results grid to two columns and adjusts the content container so that the interface can make better use of smaller screen widths. This provides basic responsiveness for different display sizes.

However, the presence of a responsive CSS rule should not be interpreted as evidence of complete mobile-device testing. The prototype should be tested on actual Android devices and different screen sizes before making claims about full mobile compatibility. The proposed Android application is intended to provide a more dedicated mobile experience rather than relying only on the browser prototype's responsive behaviour.

JavaScript provides the interactive behaviour of the prototype. It controls screen navigation, subject selection, question handling, answer processing, feedback, and result generation. JavaScript allows the application to respond to user actions without requiring a complete browser page reload.

The front-end also provides the foundation for future integration with a database and backend services. At present, the supplied browser prototype demonstrates the interface and basic workflow, while the proposed architecture can connect these interface components to persistent database records and adaptive quiz functionality.

In the proposed implementation, the front-end can communicate with the application logic to retrieve questions, submit responses, process quiz attempts, and display results. The interface can therefore be extended from a demonstration-oriented prototype into a database-driven application.

The front-end design is also intended to support future features such as topic selection, adaptive question presentation, performance summaries, weak-topic identification, learning resources, and personalized recommendations. These features can be added without changing the fundamental purpose of the examination interface.

Overall, the supplied front-end provides a basic but functional representation of the exam preparation workflow. Its single-page HTML structure, CSS styling, and JavaScript-based navigation demonstrate the core interaction between the learner and the application. The prototype serves as the user-interface foundation for the proposed Exam Preparation App with Adaptive Quizzes, while the planned

database, backend, and adaptive-learning components can extend the prototype into a more complete examination preparation system.

3.2 Back-end

The current supplied implementation is primarily a browser-based prototype and does not contain an executable backend server. The main application operations are currently performed within the browser using JavaScript. These operations include question selection, answer validation, quiz processing, grading, feedback generation, and display of result information. Therefore, the current prototype should be considered a client-side demonstration rather than a complete client-server application.

The absence of an executable backend is an important distinction in the project documentation. Although the prototype demonstrates the examination workflow, it does not currently provide a separate server process that receives requests from the browser, validates them independently, accesses a centralized database, and returns responses to the client.

The terms FastAPI, Flask, Django, and JWT may appear within internal course-outcome strings or documentation associated with the project. Their presence in such strings does not indicate that an executable FastAPI, Flask, Django, or JWT-based backend has been implemented. A backend technology can only be considered implemented when corresponding server-side source code, configuration, dependencies, endpoints, and execution behaviour are available and tested.

In the current prototype, the browser itself performs the main application logic. When a learner selects a subject, the JavaScript code can identify the corresponding questions and display them. When the learner selects an answer, the browser can compare the response with the available answer information and calculate the result. The final score can then be displayed directly through the browser interface.

This approach is suitable for demonstrating the basic workflow but has limitations when the application needs persistent, trusted, and shared data. Information calculated only on the learner's device can potentially be modified or lost. For example, a locally calculated score does not provide independent verification that the learner actually achieved that score.

The proposed Android architecture therefore separates application responsibilities into different layers. Request handling, validation, quiz processing, repository operations, and persistence are intended to be separated rather than placing all application logic inside the user interface.

In the proposed design, the application interface sends the required information to the appropriate application or service layer. The validation layer can verify whether the submitted information is valid before further processing. The quiz service can manage operations such as selecting questions, processing answers, calculating results, and maintaining the state of a quiz attempt.

The repository layer acts as an intermediate component between application services and data storage. It can coordinate access to the local Room database and, where required, cloud-based Firebase operations. This separation prevents the user interface from directly managing all database operations and makes the application easier to maintain.

The Room database is proposed for local persistence of information such as questions, subjects, learner records, quiz attempts, responses, and results. The repository can provide the required data to the application services without exposing the complete database implementation to the user interface.

Cloud operations can be coordinated through the same repository-oriented architecture when synchronization is required. The repository can determine whether information should be retrieved from local storage, written locally, synchronized with cloud storage, or processed according to the application's synchronization policy.

A server component becomes particularly important when information must be trusted independently of the learner's device. For example, if the application provides verified grades, centralized rankings, or controlled question-bank modifications, these operations should not depend entirely on calculations performed by the learner's device.

If grading is performed only on the client device, a technically capable user could potentially modify local application data or JavaScript behaviour. Similarly, a locally calculated leaderboard cannot provide a reliable ranking across multiple devices because each device may contain only a subset of the complete learner data.

A centralized backend can provide an independent environment for processing sensitive operations. It can receive quiz submissions, validate the submitted information, calculate or verify results, and store trusted records in a controlled database. The exact backend framework can be selected during implementation according to project requirements.

Authentication is another area where a backend may become necessary. If the application supports individual learner accounts, the system needs to associate quiz attempts and performance records with the correct authenticated learner.

Authentication tokens or sessions can be used to maintain the identity of the user

while communicating with backend services. However, the current prototype does not contain an implemented JWT authentication mechanism.

The proposed backend architecture can also support centralized question-bank management. Authorized users such as faculty or administrators could add, modify, or remove questions through controlled backend operations. Such changes should not depend on unrestricted editing of data stored on an individual student's device.

The backend can also provide APIs for retrieving questions and submitting quiz results. An API-based architecture would allow the Android application to communicate with backend services through defined requests and responses. Postman can be used during development to test these endpoints independently before connecting them to the Android interface.

The separation between front-end, application services, repositories, and persistence also improves maintainability. If the database technology changes in the future, the user interface does not necessarily need to be redesigned. Similarly, improvements to the adaptive quiz algorithm can be made within the quiz service without requiring major changes to the presentation layer.

The proposed architecture therefore provides a clear path from the current browser prototype toward a complete application. The current implementation demonstrates the client-side examination workflow, while the proposed backend architecture introduces the additional components required for persistent, synchronized, and trusted application behaviour.

Overall, the backend portion of the project should be documented carefully according to the actual implementation status. The supplied files currently demonstrate browser-based processing rather than an executable backend server. The proposed architecture, however, provides a foundation for introducing server-side validation, trusted grading, centralized question management, authentication, rankings, synchronization, and persistent learner records as the application is developed further.

3.3 Database

The current browser prototype uses client-side storage to maintain a limited amount of examination result information. The results are stored under the browser `localStorage` key `examLeaderboard`. This provides a simple mechanism for retaining summary information within the browser environment without requiring a separate database server.

Each stored leaderboard record contains information such as the learner's name, subject, marks, total marks, percentage, grade, and date. These values are sufficient to display a basic result summary and leaderboard within the prototype. The browser can retrieve the stored records and use them to generate the corresponding result information for the user.

However, the current storage mechanism is primarily intended for prototype demonstration. The information is stored as client-side data rather than as records in a structured relational database. As a result, the prototype does not provide the database relationships required for detailed examination history and learner-level analysis.

The question content in the current implementation is embedded directly within JavaScript arrays. Questions are therefore available to the browser as part of the application code rather than being retrieved from a database through structured queries. This approach is simple for demonstrating the quiz workflow but becomes difficult to manage when the number of questions, subjects, topics, and learners increases.

There are no SQL tables, foreign-key relationships, database migrations, or response-level records in the supplied browser implementation. The prototype therefore does not maintain a separate record for every learner response associated with a particular question and quiz attempt. Instead, it primarily maintains summary-level result information.

This limitation is important because a complete examination preparation application requires more detailed information. For example, if a learner completes multiple quizzes, the system should be able to distinguish each attempt and identify the individual questions answered during each attempt. A summary record containing only the final score cannot provide this level of information.

The proposed database design therefore introduces structured local persistence using Room. Room is an abstraction layer over SQLite and provides a structured approach for representing application data through entities, data-access objects, and a database class. It can provide the local persistence layer required by the proposed Android application.

In the proposed system, different types of information can be represented as separate entities. Learner information can be maintained separately from subject information, while question information can be maintained independently from quiz attempts and responses. This separation allows reusable information to be stored once and referenced where required.

The proposed structure can include entities representing students, subjects, questions, quizzes, quiz attempts, and responses. A student can have multiple quiz attempts, while each quiz attempt can contain multiple responses. Each response can be associated with a particular question. These relationships provide the foundation for accurate result calculation and historical performance analysis.

For example, instead of storing only a final score of 80 percent, the proposed system can preserve the individual responses that produced that score. This makes it possible to determine which questions were answered correctly, which were answered incorrectly, which topics caused difficulty, and how performance changes across different attempts.

The proposed database can also store question attributes such as subject, topic, difficulty level, question text, options, and correct answer information. These attributes can support question retrieval and adaptive selection. A query can retrieve questions belonging to a particular subject or topic and can also consider their difficulty level.

The use of structured database relationships can reduce unnecessary duplication. Question content can be stored independently and reused across multiple quiz attempts. Attempt-specific information can then reference the existing questions instead of creating a separate copy of every question for every learner.

The proposed design can also use primary keys to uniquely identify records. For example, each learner, question, subject, quiz, attempt, and response can have its own identifier. Foreign-key relationships can then connect related records. This provides a more reliable structure for retrieving information and maintaining data consistency.

Database transactions are also relevant to the proposed design. When a learner completes a quiz, the application may need to create or update an attempt record and store several associated response records. These related operations should be handled consistently so that incomplete data does not become part of the learner's permanent history.

Room can provide the local database layer for these operations. Data access objects can define the operations required by the application, while the Room database class can provide access to the underlying SQLite database. The application services can communicate with these components through the proposed repository layer.

The proposed database design also provides a foundation for SQL-based analysis. Queries can be used to calculate total attempts, average scores, correct-answer

counts, topic-level performance, and other learning statistics. Joins can combine information from students, attempts, questions, and responses, while aggregation functions can be used to calculate performance summaries.

Indexes can also be considered for fields that are frequently used for searching and filtering. For example, subject identifiers, topic identifiers, difficulty levels, learner identifiers, and attempt identifiers may be indexed depending on the final query workload. Proper indexing can improve retrieval performance as the size of the question bank and learner data increases.

The proposed design also considers the difference between local storage and cloud storage. Firebase is a separate cloud service and does not automatically convert a Room schema into a synchronized cloud database. Room manages local SQLite-based persistence, while Firebase provides its own cloud-based data services. Therefore, synchronization between Room and Firebase must be explicitly designed and implemented.

The synchronization process should define which records are stored locally, which records are synchronized with the cloud, how records are identified, and how conflicts are handled. The system should avoid assuming that local changes are automatically reflected in cloud storage.

This distinction is particularly important for learner performance and rankings. If a score is stored only in local storage, it cannot automatically be treated as a verified result across multiple devices. Similarly, a local leaderboard cannot represent a complete ranking of all learners unless the relevant records are collected, synchronized, validated, and processed through a trusted shared data source.

The proposed cloud component can therefore be used to extend the local database architecture when centralized records are required. Authentication can associate cloud records with the appropriate learner, while synchronization can allow selected information to move between local and cloud storage according to defined rules.

The current browser prototype and proposed database architecture therefore represent two different levels of implementation. The prototype demonstrates the examination workflow using localStorage and JavaScript arrays, while the proposed Android architecture introduces structured Room persistence and provides a path toward controlled cloud synchronization using Firebase.

Overall, the database design is an important part of transforming the prototype into a complete examination preparation application. Moving from summary-level local storage to structured records allows the system to maintain individual attempts and

responses, perform meaningful performance analysis, support adaptive question selection, and provide a foundation for future cloud-based functionality.

3.4 Version Control

Version control is an important part of the development of the Exam Preparation App with Adaptive Quizzes because the project contains multiple types of files, including front-end source code, Android application code, database definitions, SQL queries, configuration files, testing resources, and project documentation. Git and GitHub are listed as part of the intended development environment and can be used to maintain these resources in an organized manner.

Git provides a local version-control mechanism that records changes made to project files over time. It allows developers to maintain different versions of the source code and provides a way to identify when particular changes were introduced. This is particularly useful for a project that is developed gradually because new features, database changes, bug fixes, and interface improvements may be introduced at different stages.

GitHub can be used as a remote repository for storing the project and maintaining a centralized copy of the source code. It can also support collaboration between team members and provide a convenient location from which authorized team members can access the latest project files.

A suitable repository for this project should contain the source files required to reproduce the application. These may include the browser prototype files, Android source files, database definitions, SQL scripts, test code, configuration examples, and setup instructions. The repository should be organized so that developers can understand the purpose of each component without depending on files stored outside the project.

Database-related changes should also be maintained through version control. When the database schema is modified, the corresponding migration or schema-change files should be included in the repository. This helps ensure that the database structure used by different development environments remains consistent.

The project should maintain a clear distinction between application changes and database changes. For example, a change that introduces a new quiz feature may require application source-code modifications, while adding a new database column requires a corresponding schema update or migration. Tracking these changes separately makes it easier to identify the cause of problems during development.

No repository or commit history was supplied with the current project materials. Therefore, this report does not claim that a specific branching strategy, pull-request process, code-review workflow, or commit history has already been used. Git and GitHub are documented as suitable development and version-control tools for the project rather than as evidence of completed repository activities.

For future development, commits should be organized so that each commit represents a meaningful and understandable change. Schema changes can be committed separately from application changes where practical. This makes it easier to identify database-related modifications and allows developers to understand how the application and database have evolved.

Database releases should also be associated with migration versions. When a new table, column, relationship, index, or constraint is introduced, the corresponding database migration should have a clearly identifiable version. This is particularly important when Room is used because changes to the local database schema need to be handled through appropriate migration mechanisms.

For example, if a future version introduces a new field for question difficulty, the database schema change should be represented by an appropriate migration rather than simply modifying the table definition without considering existing data. This helps preserve existing learner records when the application is upgraded.

The repository should also contain setup instructions that explain how the project can be configured and executed. These instructions can include required development tools, SDK versions, database setup, dependency installation, build commands, environment configuration, and testing steps. Clear setup documentation makes it easier for another developer or evaluator to reproduce the project environment.

Configuration files should be handled carefully. Reusable configuration examples can be included in the repository, but actual credentials should not be committed. Service-account credentials, authentication secrets, database passwords, API keys, signing keys, and other sensitive information should remain outside version control.

For example, a configuration template can contain placeholder values showing which variables are required, without containing the real credentials used during development. This allows another developer to understand the configuration requirements without exposing private information.

Signing credentials require particular protection because they may be used to build or distribute Android applications. Keystores, passwords, and signing secrets should

not be committed to a public or shared repository unless they are protected through an appropriate secure mechanism.

The same principle applies to Firebase configuration and other cloud-service credentials. The repository can contain safe configuration examples and instructions explaining how the developer should provide the required values locally. Actual service credentials should be managed separately from the source code.

Version control also supports debugging and maintenance. If a new change introduces an error in quiz processing, database operations, or the user interface, the development history can help identify the change responsible for the problem. Earlier working versions can also be examined when troubleshooting.

For a team-based project, version control can provide additional benefits by allowing different team members to work on different components. Developers can work on the interface, database, backend, testing, or documentation and later combine their changes through an appropriate collaboration process.

As the project develops, a consistent commit naming convention can also make the history easier to understand. Commit messages should describe the purpose of the change rather than using vague descriptions. This can make it easier for team members and reviewers to understand how the application has developed.

Overall, Git and GitHub provide a suitable foundation for managing the source code and development history of the Exam Preparation App with Adaptive Quizzes. The repository should contain the necessary source files, database migrations, testing resources, documentation, and safe configuration examples while keeping confidential credentials outside version control.

The proposed version-control approach emphasizes traceability, reproducibility, database consistency, and secure configuration management. Separating schema changes from application changes where appropriate and assigning migration versions to database releases can make future development easier to manage. At the same time, avoiding the storage of real credentials in the repository helps protect the application and its associated services from unauthorized access.

3.5 APIs and External Services

Service or API	Use and current status
getUserMedia	Implemented: requests camera and microphone streams.
MediaRecorder	Implemented: starts recording and stores received chunks in memory.
YouTube links	Implemented: opens learning URLs or search results; no YouTube Data API integration.
Google sign-in	Simulated: prompts for a name and constructs an email-like string.
Firebase	Proposed: identity, cloud records, and synchronization.
Room DAOs	Proposed: local SQL access from Android code.

MediaRecorder exposes browser recording controls and data events [7]. In this prototype, recorded chunks are collected but no download or upload workflow is implemented. The report therefore describes a recording gate, not a complete proctoring service.

For a Firestore implementation, the SDK can cache and synchronize its own documents, subject to its documented offline behaviour [5]. Any bridge between that cache and Room must still be designed, implemented, and tested. Room transactions must not be described as atomic transactions spanning the device and cloud.

CHAPTER 4 SYSTEM ARCHITECTURE

4.1 High-level Architecture Diagram

Figure 4.1 represents the supplied implementation. A single browser process contains the interface and application logic. Question data is read from memory; only completed result summaries are written to browser storage.

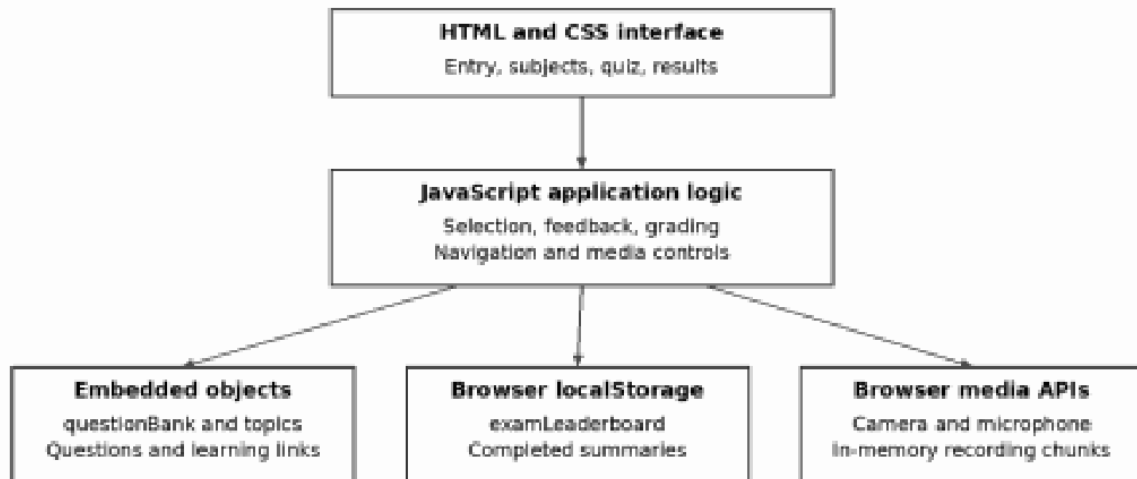


Figure 4.1 Architecture of the supplied browser prototype

The system architecture of the Exam Preparation App with Adaptive Quizzes describes the relationship between the user interface, application logic, data storage, and external services. The project currently contains a browser-based prototype that demonstrates the main examination workflow. In addition to this supplied implementation, the project proposes an Android-based architecture using Kotlin, Room, and Firebase for a more complete and persistent application.

Figure 4.1 represents the architecture of the supplied browser prototype. In the current implementation, the major application components operate inside a single browser process. The interface is implemented using HTML and CSS, while JavaScript provides the application logic responsible for navigation, question selection, answer validation, grading, and result generation.

The browser prototype follows a relatively simple architecture because it does not contain an executable backend server. The browser loads the application files and executes the JavaScript code locally.

Question information is available through JavaScript data structures, and the application logic processes the learner's interaction directly within the browser.

The question bank and topic information are represented as static JavaScript objects or arrays. When the learner selects a subject and begins a quiz, the application logic

retrieves the appropriate questions from these in-memory structures. A subset of questions can then be selected for the current quiz session.

The active quiz information is maintained within the browser while the learner is taking the examination. This includes information such as the selected subject, current question, selected answers, and quiz progress. Since these values are maintained in application memory, they represent the current state of the active quiz rather than a persistent database record.

After completion of a quiz, the application calculates the result and generates a summary. The completed result is stored using the browser's local storage mechanism under the examLeaderboard key. The stored summary contains information such as learner name, subject, marks, total marks, percentage, grade, and date.

This architecture is suitable for demonstrating the basic examination workflow because it does not require a separate database server or backend deployment. However, it also creates limitations because the stored information belongs to the current browser storage context. Different browsers or devices do not automatically share the same leaderboard information.

The proposed architecture extends this browser-based workflow into a more structured Android application. The proposed Android version separates the presentation layer, application logic, repository layer, local persistence, and cloud services. This separation is intended to improve maintainability, data consistency, scalability, and future support for adaptive learning.

In the proposed architecture, the learner interacts with the Android application through the presentation layer. The application sends requests to the appropriate application services, which process quiz-related operations and communicate with the repository layer.

The repository acts as an intermediary between the application services and the underlying data sources. It can coordinate access to the local Room database and cloud-based Firebase services. This allows the application logic to remain independent of the exact storage mechanism.

The Room database is intended to maintain local learner records, questions, quiz attempts, responses, and other information required for offline operation. When network access is available, selected records can be synchronized with cloud storage according to the defined synchronization policy.

The proposed architecture therefore provides a path from a simple browser demonstration toward a more complete application in which learner information can be stored persistently and managed through structured database relationships.

4.2 Description of Each Layer

The presentation layer provides the interface through which the learner interacts with the application. In the supplied browser prototype, HTML sections and CSS styles are used to construct the visible screens and interface components. These include the login form, subject menu, question-count selection, recording controls, quiz interface, results display, leaderboard, and topic-related learning links.

The HTML structure provides the basic organization of the interface, while CSS controls the visual presentation, spacing, layout, typography, cards, buttons, panels, and result grids. The prototype uses different sections of the same HTML document to represent different screens.

Navigation between these screens is handled using JavaScript. The showPage function controls which page section is active by removing the active state from other sections and assigning it to the selected section. This provides a multi-screen experience without requiring a complete browser page reload.

The proposed Android application would provide a similar presentation layer using Android-compatible interface components. The learner could navigate between subject selection, quiz instructions, questions, answers, feedback, and results. The exact Android interface can be implemented using the selected UI framework during the implementation stage.

The application logic layer contains the operations required to control the examination workflow. In the supplied browser prototype, JavaScript functions manage navigation, question selection, answer validation, grading, and result generation.

When a learner selects a subject, the application logic identifies the relevant questions from the available question bank. A randomized subset can be selected according to the requested question count. This allows the learner to receive a different combination of questions during different practice sessions when sufficient questions are available.

During the quiz, the selected answer is compared with the corresponding correct answer stored in the question data. The application determines whether the response is correct and provides the appropriate feedback. After the learner

completes the quiz, the application calculates the marks and percentage and determines the corresponding grade.

The application logic also creates the result information that is displayed to the learner. In the current implementation, the result summary can be stored in browser local storage for later display in the leaderboard.

In the proposed Android architecture, these responsibilities can be separated into application services. A quiz service can manage quiz creation, question selection, answer processing, and result calculation. Validation logic can verify whether submitted information is valid before it is stored.

The proposed adaptive quiz component can further extend the application logic. Instead of selecting questions only through randomization, the adaptive engine can consider learner performance and question difficulty. An IRT-based approach can be introduced when sufficient question calibration and learner-response data are available.

The data layer of the supplied prototype is based primarily on JavaScript data structures and browser storage. The question bank and topic information are represented as static JavaScript objects or arrays. These structures contain the information required to display questions and learning topics.

The active quiz state is maintained through application variables while the learner is completing the quiz. These variables represent temporary information such as the selected questions, current question position, selected answers, and other state information required during the session.

Completed result summaries are stored using the browser's `localStorage`. The `examLeaderboard` key is used to persist result summaries within the current browser storage context. This allows the prototype to display previously stored summaries after the page is reloaded, provided that the same browser storage remains available.

However, browser local storage does not provide the functionality of a relational database. The current implementation does not contain SQL tables, foreign keys, migrations, response-level records, or database transactions. Therefore, the prototype cannot provide the same level of structured historical analysis as the proposed database architecture.

The proposed Android data layer uses Room for local persistence. Room provides a structured abstraction over SQLite and can represent application information using entities, data-access objects, and a database class.

The proposed database can contain structured records for learners, subjects, questions, quizzes, quiz attempts, and responses. Relationships between these entities allow the application to preserve the connection between a learner, a particular quiz attempt, and the individual questions answered during that attempt.

The repository layer provides an important connection between the application logic and data layer. Rather than allowing every application component to access the database directly, the repository can coordinate data retrieval and storage.

For example, when the quiz service needs questions, it can request them through the repository. The repository can retrieve the required information from Room. When a learner completes a quiz, the repository can store the attempt and response records locally and later coordinate cloud synchronization when required.

The external interaction layer of the supplied browser prototype includes browser media APIs and external learning links. Media APIs can provide access to camera and microphone streams when appropriate permissions are granted by the browser. These features can support recording-related functionality included in the prototype.

The prototype also provides links to external learning resources such as YouTube. These links allow the learner to access additional learning material outside the core examination workflow. The browser navigates to the external resource rather than maintaining the external video content as part of the application's local database.

There is no application server or cloud request implemented in the supplied browser code. The prototype therefore does not depend on a remote server for its basic quiz processing.

The proposed Android architecture introduces cloud interaction as an additional layer. Firebase can be used for cloud-based records and synchronization when required. However, Firebase does not automatically synchronize a Room database. A specific synchronization mechanism must be designed and implemented.

The synchronization layer can monitor locally stored changes and maintain a queue of operations that need to be sent to the cloud. When network connectivity becomes available, pending operations can be transmitted to the appropriate cloud service.

A local transaction should be completed before the synchronization worker attempts to upload the corresponding data. This ensures that the local database contains a consistent record before the system attempts to replicate it remotely.

If network connectivity is unavailable, pending synchronization operations can remain in the local queue and be retried later. This prevents temporary network failures from immediately causing permanent data loss.

Stable attempt identifiers should be used when synchronizing quiz results. Cloud writes should be designed to be idempotent so that retrying the same operation does not create duplicate quiz attempts or duplicate results.

Immutable attempt information should also be separated from rebuildable analytics summaries. The original learner response and attempt records should remain available as the authoritative history, while performance summaries and analytical values can be recalculated when necessary.

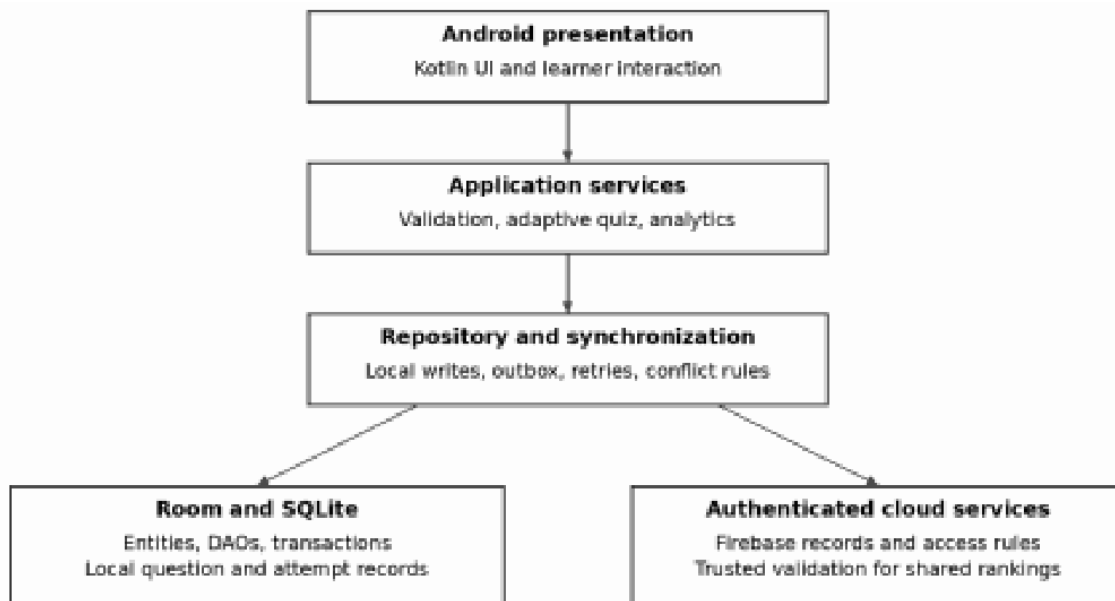


Figure 4.2 Proposed Android architecture

4.3 Deployment Architecture

The supplied browser prototype can be deployed as a static HTML-based application. For local demonstrations, a simple local HTTP server can be used to serve the HTML, CSS, JavaScript, and related resources. This provides an environment in which the browser can access the application through a local address.

The use of a local server does not create shared application storage. Browser local storage remains associated with the browser environment in which the application is executed. Therefore, two different computers or browsers will not automatically share the same examLeaderboard data.

If the application is hosted remotely and features such as camera or microphone access are required, HTTPS-based hosting is appropriate because modern browsers apply security restrictions to media access. Appropriate browser permissions must also be granted by the learner.

The proposed Android application would be installed on learner devices or executed through a supported Android emulator during development and testing. The application would contain the presentation components, application logic, repository layer, and local Room database.

The Room database would maintain the required local information, including questions, learner records, attempts, and responses. This local storage can support offline practice when the required data is already available on the device.

When network connectivity is available, a synchronization worker can process pending cloud operations. The worker can identify records that have not yet been synchronized and send them to the appropriate cloud service.

Authentication and cloud access rules should restrict learners to authorized records. A learner should only be able to access the information associated with the appropriate account or permissions. Administrative operations such as question-bank modification should be controlled separately from ordinary learner operations.

A trusted cloud process or server component becomes important when grades, rankings, or question-bank modifications must be independently verified. Calculating a grade only on the learner's device does not provide the same level of trust as server-side validation because local application data can potentially be modified.

Similarly, a shared leaderboard should be generated from a trusted centralized dataset rather than simply combining local leaderboard information from individual devices. This allows rankings to be calculated consistently across learners.

The local transaction should be completed before synchronization begins. For example, when a learner completes a quiz, the application can store the attempt and all associated responses in a local Room transaction. Once the transaction is successfully committed, the synchronization system can mark the related records as pending for cloud upload.

If the network connection fails during synchronization, the pending records should remain available for retry. The application should not delete the local record simply because the cloud upload failed.

Stable identifiers should be assigned to attempts and responses so that retrying a synchronization operation does not create duplicate records. An idempotent cloud write can check whether the record already exists before creating a new copy.

The architecture should also distinguish between immutable examination history and derived analytics. Individual attempts and responses should be treated as historical records that should not be unnecessarily modified. Summary statistics, topic

performance values, and other analytical information can be recalculated from the underlying attempt data when necessary.

This architecture provides a reliable foundation for future development of adaptive learning. As more response data becomes available, the adaptive quiz engine can analyze learner performance and question characteristics to select more appropriate questions.

The proposed architecture also supports future expansion to additional subjects, larger question banks, personalized recommendations, performance dashboards, study schedules, and other learning features. These additions can be incorporated without fundamentally changing the relationship between the presentation, application, repository, and persistence layers.

The architecture described in this chapter is therefore a combination of the supplied browser prototype and the proposed Android design. The browser architecture has been described based on the supplied implementation, while the Android, Room, Firebase, synchronization, and trusted server components represent the proposed extension of the system.

The proposed architecture has not been verified against a complete Android source-code implementation because a complete Android Studio project, Firebase configuration, and executable server implementation were not supplied with the current project materials. Therefore, the proposed architecture should be treated as a design specification for the intended implementation rather than as evidence that all proposed components are already operational.

Overall, the system architecture provides a transition from a simple client-side examination prototype to a structured adaptive examination platform. The current prototype demonstrates the user interaction and basic quiz workflow, while the proposed architecture introduces persistent database storage, modular application services, controlled synchronization, authentication, and trusted server-side processing where required.

CHAPTER 5 DESIGN

5.1 Interface and Workflow Design

The implemented user journey of the Exam Preparation App with Adaptive Quizzes is organized as a sequence of screens that guide the learner from entering basic information to completing a quiz and reviewing the final result. The workflow is designed to keep the learner focused on one stage at a time and to provide immediate feedback during the examination process.

The journey begins with the Entry screen, where the learner provides basic information such as name and email. The interface also provides a sign-in control and a simulated Google sign-in option. The information entered at this stage is used by the prototype to identify the learner when displaying quiz results and leaderboard information.

After entering the required information, the learner proceeds to the Subject screen. The prototype provides multiple subject choices, including DSA, Database, IoT, Operating Systems, German, and Japanese. Selecting a subject determines the group of questions that will be used for the subsequent quiz.

The learner then selects the required question count. The available options are 5, 8, 10, or 15 questions. The application checks the number of available questions before starting the quiz. If fewer questions are available for the selected subject, the requested number is capped according to the available question items. This prevents the application from attempting to display questions that do not exist in the selected question bank.

After the question-count selection, the learner enters the Recording screen. The supplied prototype includes recording-related controls and a video preview. The browser requests the required media permissions before the recording functionality is activated. The learner must start the recording process before continuing to the quiz according to the implemented workflow.

The recording functionality provides an additional interaction step before the examination begins. The browser media APIs are used to access the available camera and microphone streams when permission is granted. The interface displays the recording status so that the learner can identify whether the required media functionality is active.

Once the recording requirement is satisfied, the learner enters the Quiz screen. Each question is displayed individually with four answer options. The interface indicates the current question number and provides the learner with the available choices.

The learner selects one answer for each question through the interface. After the answer is selected, the application immediately checks the response against the corresponding correct answer. The feedback area communicates whether the selected answer is correct or incorrect and can display an explanation associated with the question.

The immediate feedback mechanism provides an opportunity for the learner to understand the result of the current response before continuing. Instead of allowing the learner to move immediately to the next question, the Next Question control becomes available after the feedback is displayed. This creates a clear sequence between answering the current question and moving to the next question.

The same process continues until all selected questions have been processed. The application maintains the current quiz state while the learner moves through the questions. The question number and feedback information help the learner understand the current position within the quiz.

When the final answer has been processed, the application completes the quiz workflow. The recording process is stopped and the associated media tracks are released. This prevents the camera and microphone from remaining active after the examination has finished.

The application then calculates the final performance information. The result screen provides values such as the number of correct answers, number of wrong answers, total questions, marks, percentage, and grade. The percentage is calculated from the quiz result and presented as a rounded value in the interface.

After the result has been calculated, a summary record is stored using the browser's local storage mechanism. The stored information can subsequently be used to display the learner's performance in the leaderboard. Since the current implementation uses browser storage, these records remain associated with the particular browser storage context.

The Result screen therefore acts as the completion point of the current quiz attempt. It provides the learner with an immediate summary of performance rather than requiring the learner to calculate the result manually. The result information can also be used as a basis for comparing performance across completed quizzes within the available local storage.

The learner can then move to the Leaderboard screen. The leaderboard displays stored result information such as rank, learner name, subject, marks, percentage, and grade. This provides a simple method of comparing completed results within the data available to the current browser.

The learner can also choose to start another quiz. Starting another quiz allows the learner to return to the subject and question-selection workflow and practice again. This supports repeated practice, which is one of the main purposes of an exam preparation application.

The Topics screen provides subject-related learning information and external learning links. These links can direct the learner to additional educational resources, including external video-based learning material. This extends the application beyond question answering by providing access to supplementary study resources.

The overall workflow can therefore be represented as:

Entry → Subject Selection → Question Count → Recording → Quiz → Immediate Feedback → Result → Leaderboard / New Quiz / Topics

This sequence provides a clear path through the application and reduces the number of actions required at each stage. Each screen has a specific purpose, and the learner progresses through the workflow based on the completion of the previous step.

Screen	Main controls or displayed data
Entry	Name, email, Sign In, simulated Google sign-in.
Subject	DSA, Database, IoT, Operating Systems, German, Japanese.
Question count	5, 8, 10, or 15 requested questions, capped by available items.
Recording	Video preview, permission status, recording and start controls.
Quiz	Question number, four answer options, feedback, explanation, next.
Result	Correct, wrong, total, marks, rounded percentage, grade.
Leaderboard	Rank, learner name, subject, marks, percentage, grade.
Topics	Subject headings and external learning links.

The current user journey demonstrates the main functional behaviour of the browser prototype.

It provides a complete practice cycle from learner entry through quiz completion and result review. At the same time, the workflow provides a foundation for the proposed Android application, where the same basic journey can be extended with persistent learner records, adaptive question selection, topic-level analysis, synchronized data, and personalized learning features.

The proposed adaptive version can further modify the question-selection stage according to learner performance. Instead of always relying on a randomly selected subset, the adaptive engine can use available difficulty and response information to select questions that are more appropriate for the learner's estimated ability. Such

functionality depends on having sufficient calibrated question data and learner-response history.

Overall, the implemented user journey provides a straightforward examination workflow while demonstrating the major interactions expected from the Exam Preparation App with Adaptive Quizzes. The current prototype focuses on browser-based interaction and local result storage, while the proposed architecture can extend these functions into a persistent, adaptive, and synchronized learning application.

Design Considerations

The proposed version of the Exam Preparation App with Adaptive Quizzes can improve the current user experience by making important application states and requirements more visible to the learner. The supplied prototype demonstrates the main examination workflow, but several improvements can make the system clearer, more flexible, and easier to use during repeated practice.

Before a quiz begins, the application should display the actual number of questions available for the selected subject. The current question-count options allow the learner to request a particular number of questions, but the available question bank may contain fewer questions than the requested value. Instead of only limiting the requested count internally, the proposed interface should clearly inform the learner about the available number.

For example, after selecting a subject, the interface could display a message such as “12 questions available for this subject.” The learner could then select a suitable quiz size based on the available questions. If the learner requests more questions than are available, the application should clearly explain the limitation rather than silently reducing the number.

This improvement would make the quiz-selection process more transparent and would help the learner understand the size of the available question bank. It also prevents confusion when the learner selects a larger question count but receives fewer questions than expected.

The proposed system should also provide clear information about the recording functionality before requesting camera or microphone permissions. The learner should understand why recording is required, what type of recording is being performed, when the recording starts, and when it stops.

The recording screen can display a short explanation of the recording process and the purpose of the feature. The interface should also indicate whether camera and

microphone permissions have been granted. If permission is denied, the application should provide an understandable message instead of leaving the learner uncertain about why the quiz cannot continue.

The application should clearly indicate the recording state through visible controls or status information. For example, the learner should be able to distinguish between an inactive recording state, an active recording state, and a completed recording state. This reduces the possibility of starting a quiz without realizing that the required recording has not started.

The proposed application should also explain how recordings are handled after the quiz is completed. The learner should know whether the recording is stored locally, uploaded to a server, deleted after the session, or retained for a defined period. The system should not imply that recordings are permanently stored if such functionality has not actually been implemented.

If recording storage is introduced in a future implementation, appropriate access controls should be considered. Recordings can contain sensitive personal information, so the application should restrict access to authorized users and avoid unnecessary retention. The storage policy should be clearly documented as part of the application design.

Another important improvement is navigation control while a quiz is active. Once the learner begins answering questions, the application should prevent accidental navigation that could cause the current quiz state to be lost. For example, if the learner attempts to return to the subject-selection screen while an active quiz is in progress, the application can display a confirmation message.

A confirmation message can explain that leaving the quiz may discard the current attempt. The learner can then choose either to remain in the quiz or intentionally exit it. This provides protection against accidental loss of progress.

The application should also consider what happens if the browser or Android application is closed during an active quiz. In the proposed Android version, the current attempt can be represented as a structured record so that the system can determine whether the learner has an unfinished attempt. Depending on the final requirements, the application could allow the learner to resume the attempt or explicitly mark it as abandoned.

The proposed version should also provide a practice mode in which recording is not required. The recording feature may be useful for specific assessment scenarios, but it may not be necessary for ordinary self-practice. Requiring camera or microphone

access for every practice session can make the application less convenient for learners.

A practice mode can allow learners to focus entirely on question answering and revision without enabling the camera or microphone. This would make the application more suitable for repeated study sessions, quick revision, and situations where the learner does not want to use recording features.

The application could therefore provide two clear options before starting a quiz, such as a regular practice mode and a monitored or recorded mode. The exact names can be changed according to the final project requirements. The important point is that the learner should understand the difference between the modes before beginning the quiz.

The practice mode can use the same question bank and result-processing logic as the recorded mode. The main difference would be that camera and microphone access are not required. This allows the examination preparation functionality to remain available even when media permissions are unavailable.

The proposed system should also provide clear feedback when a learner attempts to perform an action that is not currently allowed. For example, if the learner attempts to start the quiz without selecting a subject or question count, the application should identify the missing information and explain what needs to be completed.

Similarly, if the selected subject does not contain enough questions, the system should display the available count and provide suitable alternatives. These validation messages should be written in simple language so that learners can understand them immediately.

The interface should maintain a consistent navigation structure throughout the application. Buttons should have clear labels and should perform predictable actions. Important actions such as starting a quiz, submitting an answer, leaving a quiz, and completing an attempt should be visually distinguishable from ordinary navigation controls.

The proposed Android version can also improve accessibility by using readable text sizes, sufficient spacing, clear button labels, and layouts that adapt to different screen sizes. These improvements are especially important because the application is intended for repeated educational use.

The result screen can also be improved by presenting performance information in a more meaningful way. In addition to marks, percentage, and grade, the application can show the number of correct and incorrect answers and provide topic-level information when sufficient response data is available.

The application can then use the learner's previous attempts to support personalized revision. For example, repeated incorrect responses in a particular topic can be used as an indication that the learner may benefit from additional practice in that area. Such recommendations should be based on sufficient response data rather than a single incorrect answer.

The proposed improvements therefore focus on making the application more transparent, flexible, and learner-friendly. Displaying the actual question count, clearly explaining recording behaviour, protecting active quiz navigation, and providing a practice mode can address important usability limitations in the current workflow.

These changes are recommendations based on the supplied prototype behaviour rather than claims about features that are already implemented. They form part of the proposed improvements for the future version of the Exam Preparation App with Adaptive Quizzes and can be incorporated during further Android and backend development.

5.2 ER Diagram and Database Schema

The database design of the Exam Preparation App with Adaptive Quizzes is proposed to provide structured storage for learner information, subjects, topics, questions, quiz attempts, individual responses, answer options, and scheduled practice. The design is intended to overcome the limitations of the supplied browser prototype, where question content is maintained in JavaScript arrays and completed results are stored as summary information in browser storage.

Figure 5.1 represents the proposed core Entity-Relationship model. The diagram is a design proposal and is not an extracted database structure from the supplied HTML implementation. The proposed model establishes relationships between the major entities required by the application.

A Learner represents a student using the application. A learner can participate in multiple quiz attempts over time. Therefore, the relationship between Learner and QuizAttempt is one-to-many. One learner may have many attempts, while each individual attempt belongs to one learner.

A Subject represents a major examination or learning area. Each subject can contain multiple topics, and each topic can contain multiple questions. This creates a hierarchical relationship between Subject, Topic, and Question. A subject can therefore be used to organize a large question bank into meaningful learning categories.

The Topic entity provides a more detailed classification within a subject. For example, a subject may contain topics such as data structures, algorithms, database concepts, operating systems, or other relevant areas. Each topic belongs to one subject, while a subject can contain multiple topics.

The Question entity stores reusable question information. It can contain the question prompt, explanation, difficulty information, topic reference, and version information. The difficulty attribute is particularly important for the proposed adaptive quiz engine because adaptive selection requires information about question difficulty.

The QuestionOption entity stores the possible answers associated with a multiple-choice question. Each question can contain multiple options. The current application format uses four answer choices, and one of those choices should be identified as the correct answer. The relationship between Question and QuestionOption is therefore one-to-many.

A QuizAttempt represents one complete execution of a quiz by a learner. It stores information such as the learner associated with the attempt, selected subject, start time, completion time, and current status. A learner can therefore create many attempts, while every attempt belongs to one learner.

The QuestionResponse entity represents the learner's response to a particular question within a specific quiz attempt. This entity is important because it preserves the individual response history instead of storing only the final score.

The relationship between QuizAttempt and QuestionResponse is one-to-many. One attempt can contain multiple question responses, while each response belongs to one attempt. Similarly, Question and QuestionResponse have a relationship because the response identifies the question that was presented during the attempt.

The combination of attempt_id and question_id can be used as a composite primary key for QuestionResponse. This prevents the same question from being recorded more than once within the same attempt when the intended quiz format allows only one response per question.

The selected answer should also correspond to an option belonging to the same question. This relationship prevents an invalid option from being associated with a response for another question. The application should validate this condition before storing the response.

Stable identifiers are important throughout the database design. Primary keys such as learner_id, subject_id, topic_id, question_id, and attempt_id provide unique references to records. These identifiers allow records to be connected without depending on names or other values that may change.

Foreign keys preserve the relationships between entities. For example, `topic_id` in `Question` refers to the corresponding `Topic` record, while `learner_id` in `QuizAttempt` refers to the appropriate `Learner`. Foreign-key constraints can therefore help prevent records from referencing entities that do not exist.

The proposed database also supports historical examination records. `Question` content may change during the lifetime of the application. For example, an explanation may be corrected, an option may be modified, or the answer key may be updated. If the original question is changed without preserving its earlier version, historical attempts may become difficult to interpret.

To address this issue, the `Question` entity includes a version field. When a question or its answer key changes, an appropriate version of the question should be retained for historical attempts. This ensures that a previous attempt can still be interpreted according to the question that was actually presented at that time.

A completed `QuizAttempt` should normally be treated as immutable. Once an attempt has been completed, its questions, responses, marks, and other historical information should not be modified during normal application operation. If an administrative correction is necessary, it should be performed through a controlled process and recorded using an audit mechanism.

This approach is important for maintaining trustworthy examination history. If completed records could be freely changed, the learner's historical performance would not provide a reliable representation of previous quiz activity.

The proposed design also includes validation rules for quiz attempts. For example, a question selected for an attempt should belong to the subject associated with that attempt. This prevents a question from an unrelated subject from accidentally being included in the quiz.

Simple foreign keys may not always be sufficient to enforce cross-table rules of this type. Therefore, such conditions can be checked within the application transaction or through appropriate database-level mechanisms such as triggers where supported.

Transactions are important when creating and completing quiz attempts. When a learner completes a quiz, multiple related records may need to be inserted or updated. The attempt record and its associated responses should be committed consistently so that the database does not contain a partially completed result.

If an error occurs during the operation, the transaction can be rolled back according to the database mechanism. This provides stronger consistency than independently storing each response without transaction control.

The database design also provides a foundation for performance analysis. Since every response is connected to an attempt, question, topic, and learner, the application can calculate statistics such as total attempts, average marks, correct-answer percentage, topic-level accuracy, and improvement over time.

For example, the application can identify the number of questions answered correctly within a particular topic. It can also compare performance between earlier and later attempts. These calculations provide more meaningful learning information than a single final percentage.

The proposed structure can also support the adaptive quiz engine. Question difficulty can be stored as part of the Question entity, while learner response history is preserved through QuestionResponse. The adaptive engine can use this information to select questions that are appropriate for the learner's current performance level.

An adaptive system should not classify a learner as weak in a topic based on one incorrect response. Instead, sufficient response history should be considered before generating topic-level recommendations. This helps reduce misleading conclusions caused by isolated mistakes.

The MockSchedule entity extends the database design to support planned practice. It can store a learner's scheduled subject, planned date and time, and status of the scheduled practice session. This provides a foundation for future reminder and study-planning functionality.

The schedule information can also be used to support personalized practice plans. For example, the application could identify topics that require revision and allow the learner to schedule future practice sessions. The exact scheduling and notification mechanisms can be introduced during later implementation stages.

The proposed schema is therefore designed to separate reusable question information from attempt-specific information. Question content can be reused across multiple attempts, while responses remain associated with the specific learner and attempt in which they occurred.

This separation reduces unnecessary duplication and makes the database easier to maintain. It also provides a clearer basis for SQL queries, joins, aggregation, indexing, and analytical processing.

SQLite-compatible SQL examples for the proposed database structure can be included in Appendix B. These examples can demonstrate table creation, primary keys, foreign keys, constraints, indexes, and sample retrieval queries. They represent the proposed schema and should be tested before being integrated into the Room database implementation.

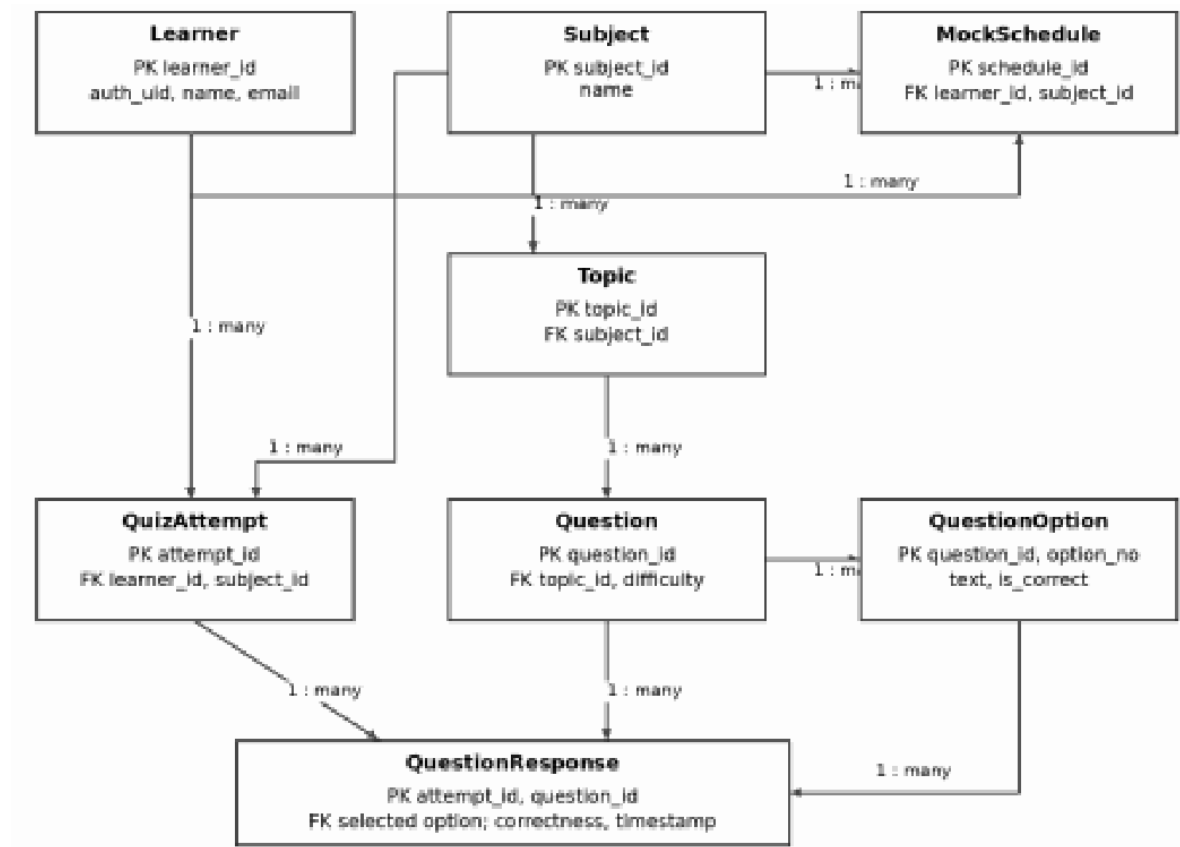


Figure 5.1 Proposed ER model with one-to-many relationships

The Learner entity forms the identity layer of the proposed database. The learner_id provides an internal stable identifier, while auth_uid can connect the record with the authentication system when cloud authentication is implemented. The learner's name and email are stored as profile information.

The Subject entity provides a controlled list of available examination areas. Keeping subjects in a separate table prevents the application from repeatedly storing subject names with every question. A unique constraint on the subject name can prevent duplicate subject records.

The Topic entity creates a second level of organization. The subject_id foreign key connects each topic to its parent subject. This makes it possible to retrieve all topics belonging to a selected subject and all questions associated with a particular topic.

The Question entity is one of the most important entities because it represents the reusable content of the question bank. The difficulty field can be used by the adaptive quiz engine, while the version field protects the interpretation of historical quiz attempts.

The QuestionOption entity separates answer choices from the main question record. This design allows the number and content of answer options to be managed

independently. For the current multiple-choice format, four options can be associated with each question.

The QuizAttempt entity records the learner's execution of a quiz. The status field can represent stages such as started, completed, abandoned, or another controlled state defined by the final implementation. Status transitions should be validated so that invalid state changes do not occur.

The QuestionResponse entity stores the actual learner interaction with a question. It records which option was selected, whether the answer was correct, and when the response was submitted. This provides detailed historical information that is not available in the current browser localStorage summary.

The MockSchedule entity supports future study planning. It connects a learner and subject with a scheduled practice time and status. It can later be extended to support reminders, recurring practice, or personalized study plans.

Overall, the proposed ER model provides a normalized structure for the Exam Preparation App with Adaptive Quizzes. It separates learner identity, subject organization, question content, answer options, quiz attempts, individual responses, and scheduled practice. This structure supports reliable relationships, historical records, SQL analysis, adaptive question selection, and future synchronization with cloud services.

The design also provides a clear transition from the current browser prototype to the proposed database-driven application. While the supplied prototype uses JavaScript arrays and browser storage, the proposed schema introduces persistent structured records and explicit relationships that can support a larger number of learners, questions, attempts, and performance records.

Proposed Data Dictionary

Entity	Key fields	Purpose and constraints
Learner	learner_id PK; auth_uid unique; name; email	One stable identity per account.
Subject	subject_id PK; name unique	Subject catalogue.
Topic	topic_id PK; subject_id FK; name	Unique topic name within a subject.
Question	question_id PK; topic_id FK; prompt; explanation; difficulty; version	Difficulty is required for the proposed adaptive engine.
QuestionOption	question_id FK + option_no PK; text; is_correct	Four options for the current MCQ format; one correct option validated.
QuizAttempt	attempt_id PK; learner_id FK; subject_id FK; started_at; completed_at; status	Stable ID; only valid status transitions permitted.
QuestionResponse	attempt_id FK + question_id FK PK; selected_option; is_correct; answered_at	Unique question response per attempt; option belongs to question.
MockSchedule	schedule_id PK; learner_id FK; subject_id FK; scheduled_at; status	Proposed schedule and reminder data.

5.3 Normalization and Integrity

First normal form is supported by storing one option and one response per row rather than comma-separated values. Second normal form is supported by keeping attempt-wide data in QuizAttempt and question content in Question; response attributes depend on the whole attempt question key. Third normal form is supported by storing subject and topic descriptions separately from attempts and responses.

Scores and topic accuracy can be derived from response rows. If cached summaries are added, treat them as rebuildable projections with explicit update rules. The full functional dependencies and constraints must be verified during implementation; an ER diagram alone does not prove normalization.

CHAPTER 6 IMPLEMENTATION

6.1 Module-wise Implementation

Module	Implemented logic
Navigation	showPage switches the active screen.
Learner entry	login checks nonempty name and email; googleLogin uses a prompt.
Question selection	selectSubject and selectCount choose a subject and randomized subset.
Recording	startRecording requests streams and starts MediaRecorder.
Quiz rendering	displayQuestion creates option buttons and resets feedback visibility.
Answer evaluation	checkAnswer compares the selected index with the answer key.
Result processing	showResult derives marks, rounded percentage, and grade.
Storage and ranking	saveResult appends and sorts summaries; loadLeaderboard renders them.
Learning resources	loadTopics creates links from the embedded topic catalogue.

6.2 Front-end Logic and State Management

The variables `currentStudent`, `currentEmail`, `selectedSubject`, `selectedCount`, `questions`, `currentQuestion`, `correctAnswers`, and `wrongAnswers` hold the current session. The selected subset is chosen before the quiz starts, and answering a question does not change the remaining selection.

Each option button calls `checkAnswer`. The function disables all options on that screen, increments the correct or wrong counter, and displays the question explanation. For a wrong answer, it also constructs a YouTube search link from the question text. `nextQuestion` advances the index or calls `finishQuiz`.

Because state is held in memory, reloading the page loses the active attempt. Only a completed result is saved. A future implementation should persist enough attempt state to recover from interruption without double-counting answers.

Scoring and Grading

Marks equal the number of correct answers. Percentage is `Math.round(correctAnswers / questions.length * 100)`. The grading rules are applied to that rounded percentage, so the report uses the same boundaries as the code.

Percentage after rounding	Grade
90 to 100	A+
80 to 89	A
70 to 79	B
60 to 69	C
50 to 59	D
Below 50	F

6.3 API Integration

The implemented integrations are browser camera and microphone access and external learning links. No Firebase SDK initialization, HTTP application API, Room DAO, or SQL driver is present. Postman tests cannot validate a custom backend that is not supplied.

For the proposed Android version, the quiz service would retrieve eligible questions through a repository, persist answers inside a local transaction, and enqueue authenticated cloud updates. The interface contract should define stable IDs, validation errors, retry behaviour, and what happens when a submitted response is received twice.

6.4 Authentication and Authorization

The current entry screen identifies a learner for demonstration; it does not verify an identity. The Google-labelled button creates an email-like string from a prompted name. Header navigation is also available without an authenticated session.

The proposed version should use verified account authentication and enforce ownership at the data-access boundary. Restrict question-bank editing to authorized maintainers. A shared ranking should use trusted validated results, because browser data and answer keys can be modified by the user.

CHAPTER 7 FEATURES

7.1 List of Main Features

Feature	Current supplied prototype	Proposed extension
Practice quizzes	Random subject-based selection	IRT-based item selection
Feedback	Immediate correct/wrong feedback and explanations	Reviewed explanations and broader content
Performance	Attempt score and grade	Trends and topic-level response analysis
Weak topics	Not calculated	Accuracy and evidence thresholds per topic
Leaderboard	Browser-local attempt summaries	Comparable verified cohort ranking
Mock scheduling	Not implemented	Planned attempts and reminders
Persistence	Local result summaries	Room records and cloud synchronization
Recording	Required media gate; in-memory chunks	Explicit retention and lifecycle policy

7.2 How Each Feature Works

The learner selects a subject from the available options, such as DSA, Database, IoT, Operating Systems, German, or Japanese, and specifies the required number of questions. The application retrieves questions from the available question bank and generates a quiz based on the selected subject. In the current prototype, questions are selected randomly, and the requested question count is limited by the number of questions available in the question bank. Therefore, requesting 15 questions does not necessarily guarantee that all 15 questions will be displayed.

In the proposed extension, Item Response Theory (IRT) will be introduced to improve question selection. The system will estimate the learner's ability based on previous responses and select subsequent questions according to their difficulty and information value. This approach aims to provide a personalized assessment experience instead of relying only on random question selection.

The application provides immediate feedback after the learner submits an answer. It indicates whether the selected answer is correct or incorrect. When an incorrect answer is submitted, the system displays the correct answer along with an explanation and a relevant YouTube learning resource. This helps learners understand their mistakes and revise the corresponding concepts. The proposed

extension will further introduce reviewed explanations and broader learning materials to support conceptual understanding.

After completing the quiz, the application displays the learner's performance through a result page containing the total number of questions, correct answers, incorrect answers, marks obtained, percentage, and final grade. These details help learners understand their performance in the completed assessment. The proposed extension will include performance trends across multiple attempts and topic-level response analysis to monitor improvement over time.

In the current prototype, weak topics are not automatically identified. The proposed extension will calculate accuracy separately for each topic by comparing correct answers with the total attempted questions. For example, if a learner answers 4 out of 10 questions correctly, the accuracy is 40%. The system will apply predefined accuracy thresholds and minimum evidence requirements before identifying a topic as weak. This will reduce the possibility of classifying a topic based on insufficient responses.

The current leaderboard stores attempt summaries in browser local storage and sorts them according to percentage in descending order. However, it combines different subjects and question counts and may display multiple attempts from the same learner. Therefore, it represents an ordering of locally stored attempts rather than a verified comparison of student performance. The proposed extension will introduce authenticated student records and centralized cohort-based rankings using comparable assessment conditions.

Mock test scheduling is not implemented in the current prototype. The proposed extension will allow learners to plan assessments by selecting a subject, date, and preferred time. The system will maintain scheduled attempts and provide reminders before the examination begins. This functionality will support structured preparation and help students manage their study schedules.

The current prototype stores result summaries locally in the browser. Although this allows students to access previously saved attempts within the same browser environment, it does not provide centralized access across different devices. The proposed extension will introduce database-based persistence and cloud synchronization to maintain student details, quiz attempts, marks, grades, and assessment records. This will provide consistent access to assessment history across supported devices.

The current prototype includes a mandatory camera and microphone permission gate before starting the quiz. It captures video and audio data as in-memory

recording chunks during the assessment and automatically stops recording after the quiz is completed. However, it does not implement a complete media retention and lifecycle management policy. The proposed extension will define recording consent, storage duration, access permissions, and deletion procedures to ensure responsible handling of recorded information.

The proposed adaptive learning mechanism will update the learner's estimated ability after each response and select suitable questions from the available question bank. The selection process will consider question difficulty, discrimination, topic coverage, and question exposure limits. Before implementing adaptation claims, the system will require calibrated questions and evaluation of the stability of ability estimates.

Overall, the proposed extension aims to transform the existing subject-based quiz prototype into an adaptive learning and assessment system. It will provide personalized question selection, detailed performance analysis, weak-topic identification, improved feedback, scheduled assessments, centralized student records, and clearly defined recording management. These enhancements will support continuous learning and provide a foundation for future development of advanced educational assessment functionalities.

CHAPTER 8 TESTING

8.1 Front-end and Backend Testing

Testing of the available project focuses on the supplied HTML file and its associated JavaScript functionality. The testing process involves examining the source code, identifying implemented features, verifying user interactions, and checking how assessment information is processed and stored. Code inspection helps determine the existing application structure, implemented functions, local storage mechanisms, and integrations that are not yet available.

Front-end testing verifies whether the application responds correctly to user actions, including entering learner details, selecting subjects, choosing the number of questions, granting camera and microphone permissions, attempting quizzes, submitting answers, and viewing results. It also examines whether correct and incorrect answers are evaluated properly and whether explanations and learning resources are displayed as expected.

Functional testing is performed using synthetic learner information to avoid using actual student records. Different input conditions are considered to examine the application's behaviour, including empty fields, valid entries, unavailable questions, incorrect answers, and attempts to start the quiz without completing the mandatory recording process.

Backend testing is considered part of the proposed system because the supplied prototype primarily operates through HTML and JavaScript without a fully integrated backend service. The proposed backend will be responsible for processing learner requests, validating submitted information, managing quiz attempts, storing assessment records, and retrieving student performance information.

Postman will be used as a testing tool after the backend APIs or callable services are implemented. It will help verify whether the application communicates correctly with the backend and whether the services return the expected responses for different requests.

The proposed Postman testing will include submitting valid and invalid request payloads, checking missing or incorrect parameters, verifying authentication requirements, and testing access restrictions between different learner accounts. It will also examine duplicate quiz submissions, invalid question identifiers, and attempts to modify previously completed assessment records.

The backend response will be checked for appropriate HTTP status codes, meaningful error messages, and consistent response structures. Successful

requests should return the expected assessment information, whereas invalid requests should be rejected with suitable error responses.

Database-related testing will verify whether learner details, quiz responses, marks, grades, and attempt summaries are stored and retrieved correctly. It will also examine whether duplicate records are prevented and whether the application maintains consistency when a request fails during processing.

Security testing will focus on authentication and authorization mechanisms to ensure that learners cannot access another learner's private assessment records. Invalid authentication credentials, expired sessions, and unauthorized requests will be included in the proposed testing procedure.

The testing scope also distinguishes between currently implemented functionality and future integrations. Successful browser-based functional checks do not establish the correctness of Android application behaviour, Firebase security rules, cloud synchronization, or Item Response Theory calculations. These components require separate implementation and validation.

No Postman execution results are claimed in this report because the corresponding backend services have not yet been fully implemented. The proposed API testing activities will be carried out during backend integration, and their actual results will be documented after execution.

8.2 Integration Testing

The integration testing process focuses on verifying whether the different modules of the Exam Preparation Application work together correctly throughout the complete assessment workflow. The central integration path begins with subject selection, followed by selecting the number of questions, completing the mandatory recording permission process, attempting the quiz, evaluating submitted answers, generating the final result, storing the assessment summary, and displaying the leaderboard. Each stage is checked to ensure that the information passed between the modules is accurate and consistent.

The subject selection module is tested to verify whether the selected subject is correctly maintained when the learner proceeds to the question selection stage. The question count module is checked to ensure that the requested number of questions does not exceed the available question bank. The recording gate is then tested to confirm that learners cannot begin the quiz without granting the required camera and microphone permissions and successfully starting the recording process.

During quiz integration testing, synthetic learner responses are used to verify whether the answer evaluation module correctly identifies right and wrong answers. The result generation module is checked to ensure that the number of correct answers, incorrect answers, marks, percentage, and grade correspond to the submitted responses. The generated assessment summary is then passed to the storage module and retrieved for leaderboard rendering.

Recording integration tests use synthetic camera and microphone media instead of personal recordings. These tests verify whether the recording process starts before the quiz, continues during the assessment, and stops automatically after the final question. They also examine whether the application handles recording interruptions and permission failures appropriately.

Browser persistence testing is performed by completing a sample quiz and checking whether the generated assessment summary remains available after refreshing the page or restarting the browser. This helps verify whether the local storage mechanism correctly preserves completed assessment information. The test also examines whether previously stored summaries are displayed without creating unnecessary duplicate entries.

For the proposed Room database and cloud integration, additional tests will be performed to examine data consistency during network interruptions. The application will be disconnected from the network while a learner is attempting a quiz. After restarting the application and reconnecting to the network, the system should synchronize the saved responses and final assessment record correctly.

Synchronization testing will verify that repeated requests do not create duplicate records and that each response and completed attempt is synchronized exactly once. It will also examine whether incomplete attempts are handled appropriately when the application is interrupted before final submission.

Multi-device integration testing will involve two devices submitting different quiz attempts for the same learner account. The system should maintain both assessment records independently without overwriting one attempt with another. Each attempt should retain its corresponding subject, questions, responses, marks, and submission details.

Question version testing will verify how the application handles outdated questions when the question bank is updated during an assessment. The system should maintain consistency between the questions presented to the learner and the responses submitted for evaluation. This will help prevent incorrect result calculations caused by changes in question versions.

Authorization testing will verify whether learners can access only their permitted assessment records. Attempts to retrieve or modify another learner's information through unauthorized requests should be rejected. These checks will be conducted after implementing the proposed authentication and cloud security mechanisms.

Overall, integration testing will ensure that the individual modules operate together as a complete assessment workflow. The current browser-based checks focus on the available prototype, while Room database synchronization, multi-device consistency, cloud integration, and advanced security testing remain future validation activities.

8.3 Tools Used for Testing

Source code inspection was performed using the supplied HTML file to identify the implemented functionalities, JavaScript operations, local storage mechanisms, and missing backend integrations. The inspection helped determine the existing application structure and understand how different modules interact during the quiz assessment process.

Functional verification was carried out using Chromium controlled through Playwright to examine the application's behaviour under different user interactions. The verification covered the major workflow, including learner entry, subject selection, question count selection, mandatory recording permissions, quiz initiation, answer submission, result generation, and storage of assessment summaries.

Synthetic camera and microphone inputs were used to test the mandatory recording gate without capturing personal recordings. This approach allowed the recording functionality to be examined using demonstration media while maintaining the privacy of actual learners.

Screenshot captures were generated using demonstration learner information and sample quiz responses. These screenshots provide visual evidence of the application's interface and the tested workflow, including subject selection, quiz questions, feedback, result generation, and learning resources.

The testing scope was limited to the supplied prototype and the available browser-based functionality. The actual observed results of the functional verification are presented in the following test case table. These results represent the specific demonstration environment and should not be interpreted as proof of complete system validation across all platforms.

Android instrumentation testing, Room database migration testing, Firebase security rule verification, and concurrency or load testing are planned for future development

stages. These activities will be necessary to validate the behaviour of the proposed Android and cloud-based implementation.

No performance benchmark, large-scale user study, or measurable improvement in examination outcomes is asserted. Such conclusions would require separate experimental evaluation using appropriate datasets, test conditions, and participant groups.

8.4 Test Cases and Results

The functional testing of the Exam Preparation Application was conducted to verify the correctness of its implemented features and the interaction between different modules. The test cases cover form validation, learner navigation, question selection, mandatory recording, answer evaluation, result generation, data persistence, and learning resource display.

The following table presents the expected behaviour and observed results obtained during functional verification.

ID	Check	Expected result	Observed
T01	Submit empty entry form	Nonempty-field error; stay on form	PASS
T02	Enter demo name and email	Open subject selection	PASS
T03	Request 15 Database items	Cap at available bank size	8 questions
T04	Start before recording	Block quiz with alert	PASS
T05	Start synthetic media	Enable quiz entry	PASS
T06	Answer 4 of 5 correctly	4/5, 80%, grade A	PASS
T07	Reload after completion	Summary remains in storage	1 stored attempt
T08	Open topics screen	Render embedded learning cards	20 cards

The test results indicate that the implemented prototype supports the major assessment workflow in the tested browser environment. The question selection test confirmed that the requested number of questions was restricted according to the available question bank. The recording gate prevented quiz entry before recording was initiated, while synthetic media successfully enabled the assessment process.

The answer evaluation test confirmed the expected score calculation for the demonstration attempt. Similarly, the persistence test verified that one completed assessment summary remained available after reloading the application. The topics screen successfully displayed 20 embedded learning cards.

These results represent limited functional verification performed using Chromium controlled through Playwright. They do not establish complete compatibility across

different devices, browsers, or operating systems. Permission-denial scenarios, storage failures, accessibility requirements, and concurrent browser sessions require additional testing.

The screenshots presented in Appendix A were captured using the same demonstration workflow and provide visual evidence of the tested application functionalities.

Additional Acceptance Tests for the Proposed System

The following acceptance conditions are defined for the proposed backend, database, adaptive learning, and cloud synchronization functionalities. These tests will be applied after implementation.

Area	Required acceptance condition
Database integrity	Foreign-key violations and duplicate responses are rejected.
Transaction rollback	Interrupted completion leaves no partially finalized attempt.
Adaptive selection	The next item responds to the updated learner estimate and content rules.
Synchronization	Retries preserve one cloud record per attempt and do not overwrite unrelated attempts.
Authorization	Learners cannot read or change another learner's protected records.

The proposed acceptance tests will help evaluate the reliability, consistency, security, and correctness of the integrated system. Their actual outcomes will be recorded only after implementation and execution.

Android instrumentation tests, Room database migration tests, Firebase security rule verification, and concurrency or load testing remain future validation activities. No performance benchmark, large-scale user study, or improvement in examination outcomes is claimed based on the current functional testing.

CHAPTER 9 DEPLOYMENT

9.1 Steps to Deploy

The deployment process begins by preserving a copy of the original supplied HTML file to ensure that the existing prototype remains available for reference and future modifications. A separate project folder should be created to organize the application files. The supplied HTML file should then be copied into this folder and renamed as `index.html` so that it can be accessed easily through a local web server.

For local demonstration, a Python HTTP server can be started from the project directory using the following command:

```
python -m http.server 8000
```

After starting the server, open the following address in a compatible web browser:

```
http://localhost:8000
```

The browser will display the application interface, allowing the learner to interact with the available features. The user can enter demonstration name and email information, select a subject, specify the number of questions, and proceed through the mandatory camera and microphone permission process.

After granting the required permissions, the learner can attempt the quiz, submit answers, view immediate feedback, and complete the assessment. The result page should be checked to verify the displayed marks, percentage, and grade. The leaderboard should also be examined to confirm whether the completed attempt is reflected in the locally stored assessment summaries.

For remote demonstrations, the static HTML file can be uploaded to a hosting platform that supports HTTPS. After deployment, the application should be tested again using the hosted URL. Camera and microphone permissions, external learning links, and browser storage behaviour must be verified separately because these may differ between local and hosted environments.

The local HTTP server command is provided as a setup instruction for demonstration purposes and does not indicate that the application has been publicly deployed.

9.2 Environment Configuration

The supplied browser-based prototype does not require API credentials, external environment variables, or a separate configuration file. Its primary execution requirements include a compatible web browser, a suitable local HTTP server, and permission to access the camera and microphone.

Camera and microphone functionality depends on browser security restrictions and user permissions. HTTPS or an eligible secure context is required for media access when accessing the application remotely. If the learner denies the required permissions, the application should display an appropriate message and prevent the recording-dependent quiz workflow from proceeding.

External YouTube learning resources require an active internet connection. Without network access, embedded videos and external educational links may not load correctly.

The application uses browser-local storage to maintain assessment summaries and leaderboard information. These records are associated with the browser storage context and origin. Therefore, data saved while running the application locally may not automatically appear when the application is accessed through another browser, a different localhost configuration, or a hosted domain.

The proposed Android implementation will require additional environment configuration, including the application ID, Android SDK version, dependency versions, database configuration, and Firebase project settings. The Room database will require a defined schema version and migration strategy to support future database changes.

Development and production environments should be maintained separately to prevent demonstration data from mixing with actual learner records. Firebase authentication, database access rules, and cloud configuration must be verified before storing real student information.

Sensitive configuration values should not be exposed in publicly distributed source code. Access permissions must be reviewed to ensure that only authorized users and application components can access protected assessment information.

9.3 Hosting of Frontend and Backend

The supplied project currently consists of a browser-based frontend implemented using HTML, CSS, and JavaScript. Therefore, the existing prototype requires static file hosting and does not require a separate backend deployment.

For local demonstrations, the application can be accessed through a local HTTP server. For remote access, the frontend can be hosted on a static hosting platform that supports HTTPS. The hosted application should be tested to verify that its navigation, quiz functionality, recording permissions, local storage, and external learning resources operate correctly.

The supplied project files do not contain a fully implemented backend service. Therefore, no backend deployment can be claimed for the current prototype. The existing browser-based application should be distinguished from the proposed Android and cloud-based architecture.

The proposed system will require separate deployment of the Android application and the cloud components that are actually implemented. The Android application can be packaged and distributed through an appropriate installation mechanism, while backend services can be deployed using the selected cloud infrastructure.

The proposed backend architecture may include authentication services, database access, quiz management, result processing, and synchronization functionalities. Each component should be deployed and verified independently before integrating it with the complete application.

Database deployment should include the initial schema, version management, and migration procedures. The deployment process should also maintain appropriate backup and recovery mechanisms to protect learner assessment records.

Before releasing an updated version, the development team should record the application version, configuration changes, database migration requirements, and rollback procedure. This will help identify deployment-related issues and restore a previously working version if necessary.

The current project does not include a verified live production URL or a signed Android application package. Therefore, the deployment described above represents the procedure for running the existing prototype and the planned deployment approach for the proposed extended system.

The supplied Exam Preparation Application currently consists of a frontend developed using HTML, CSS, and JavaScript. The application can be hosted using a static web hosting service without requiring a separate backend server. For local demonstration, the frontend can also be executed using a local HTTP server through a compatible web browser.

The uploaded project files do not contain a fully implemented backend service. Therefore, backend deployment is not applicable to the current prototype. The existing application primarily handles user interactions, quiz execution, answer evaluation, result generation, and local result storage within the browser environment.

For the proposed extended system, the frontend will be developed as an Android application and distributed through an appropriate Android application package. The backend components will be deployed separately based on the implemented

architecture. These components may include authentication services, database operations, quiz management, result processing, and cloud synchronization.

The proposed system will use Room for local database management and Firebase or an equivalent cloud platform for centralized data storage and synchronization. The frontend and backend will communicate through the configured services to maintain consistency between locally stored assessment records and cloud-based learner information.

During deployment, the application release version, database schema version, migration procedure, and rollback strategy must be documented. Database migrations should preserve existing learner records and prevent data loss during application updates. A rollback procedure should also be maintained to restore the previous stable application version in case of deployment-related failures.

The deployment process will include functional verification of the application, database connectivity, authentication, data synchronization, and access permissions before making the system available to learners.

At the current stage, no live production hosting URL or signed Android application package has been supplied. Therefore, the proposed Android and backend deployment activities are considered future implementation work rather than completed deployment activities.

CHAPTER 10 CHALLENGES AND LIMITATIONS

10.1 Issues Identified During Review

The major challenge identified during the project review is the difference between the proposed adaptive Android-based assessment system and the currently supplied browser-based prototype. The existing HTML application uses randomized question selection, in-memory application state, and browser-local storage. Although these features support the basic quiz demonstration, they are insufficient to demonstrate advanced functionalities such as Item Response Theory (IRT) calibration, relational database constraints, ACID transactions, and synchronization across multiple devices.

Another limitation identified during code inspection is the use of simulated sign-in functionality. The current implementation does not provide complete authentication and authorization mechanisms. Additionally, unrestricted header navigation may allow learners to access application sections without following the intended assessment workflow.

The leaderboard implementation also requires security improvements. Learner names are rendered using innerHTML, which may introduce HTML injection risks when untrusted information is displayed. Using safe text-rendering methods such as `textContent` would help prevent unintended HTML interpretation.

Furthermore, the current leaderboard provides a clearing option without implementing administrative role verification. In a multi-user environment, this operation should be restricted to authorized users to prevent accidental or unauthorized deletion of assessment records.

The application also lacks centralized data management, which limits the ability to maintain consistent learner records and compare performance across different devices. These issues indicate the need for additional development before the application can be considered a complete adaptive assessment platform.

10.2 Solutions Applied and Recommended

The supplied prototype already includes several controls that support the basic assessment workflow. The question selection mechanism limits the requested number of questions according to the available question bank. Answer buttons are disabled after selection to prevent repeated submissions for the same question.

The application also stops media recording after normal quiz completion and displays a confirmation message before clearing stored leaderboard results. These

implemented controls improve the consistency of the demonstration process and reduce unintended user actions.

However, additional improvements are required to address the identified limitations. Learner information should be rendered using safe text-based methods instead of directly inserting untrusted values into HTML elements. This will reduce the possibility of HTML injection through learner-provided information.

The random sorting method used for question shuffling should be replaced with an unbiased Fisher–Yates shuffle algorithm. This will provide a more reliable random ordering of questions.

Storage operations should also include error handling for unavailable, malformed, or corrupted browser storage. The application should display meaningful messages when assessment results cannot be saved or retrieved.

Media management requires additional improvements to ensure that camera and microphone streams are properly released when the learner completes the quiz, exits the application, or encounters an interruption. The recording data collection array should be reset before starting a new recording session to prevent previous recording chunks from being included accidentally.

A clear recording retention policy should also be introduced to explain whether recorded media is stored, exported, uploaded, or deleted after the assessment.

The learning resource functionality should be updated so that previous YouTube search links are cleared when a new answer is evaluated. This will prevent a learning link associated with an earlier incorrect response from remaining visible after a correct answer.

The source HTML was not modified during the preparation of this report. Therefore, the above recommendations represent proposed improvements rather than completed code modifications.

10.3 Current Limitations or Known Risks

The current question bank does not contain calibrated difficulty information for each question. As a result, the application cannot estimate learner ability using an established IRT model or adjust question selection according to individual performance.

The prototype does not maintain a complete response history for every assessment. Consequently, detailed analysis of learning progress, topic-level accuracy, and performance changes across multiple attempts is limited.

Another limitation is the absence of quiz recovery functionality. If the browser is closed or the application is interrupted during an assessment, the learner cannot reliably resume the incomplete quiz from the previous question.

The recording functionality maintains media chunks in memory without implementing a complete export or upload mechanism. Therefore, recorded media is not available as a persistent assessment record. Starting another recording without properly stopping an existing media stream may also leave resources active.

The application has no verified learner population, standardized assessment form, or calibrated question difficulty framework for establishing comparable student rankings. The existing leaderboard therefore represents locally stored attempt percentages rather than a validated measure of learner ability.

Additional limitations include the absence of verified response-time measurements, cloud synchronization results, security testing, and learning-effectiveness evaluation. These limitations must be considered when interpreting the current prototype and presenting its capabilities during project evaluation.

Overall, the identified challenges provide a foundation for improving the application architecture, data management, security, recording functionality, and adaptive assessment mechanisms in future development stages.

CHAPTER 11 FUTURE ENHANCEMENTS

11.1 Planned Features

The first planned enhancement is the development of a persistent Android-based quiz workflow. The proposed implementation will use Room database entities and Data Access Objects (DAOs) to manage learner information, questions, quiz attempts, and individual responses. This will allow the application to maintain assessment records beyond a single browser session and provide the foundation for future performance analysis.

The application will also introduce incomplete assessment recovery. When a learner is interrupted during a quiz, the system should preserve the necessary attempt information and allow the learner to resume the assessment according to the defined recovery rules. Local database consistency should be verified before introducing cloud synchronization.

The next enhancement will focus on topic classification and question difficulty metadata. Each question will be associated with a subject, topic, and reviewed difficulty level. This information will support topic-level performance analysis and identification of areas requiring additional practice.

Weak-topic identification will be implemented using accuracy calculations and minimum evidence thresholds. Instead of identifying a topic as weak based on a small number of responses, the system will consider the total number of attempted questions and the corresponding accuracy percentage. The learner will also be shown the number of responses used to calculate each topic-level estimate.

Mock-test scheduling and reminder functionality will be introduced to support structured examination preparation. Learners will be able to schedule assessments, receive reminders, and distinguish between regular practice sessions and timed mock examinations.

Item Response Theory (IRT) adaptation will be implemented after preparing and reviewing the question bank. The development process will include selecting an appropriate IRT model, defining question calibration requirements, and establishing procedures for estimating learner ability.

Special attention will be given to new learners and newly introduced questions because sufficient response data may not initially be available. The model and calibration process will be documented and evaluated using a suitable response dataset before adaptive assessment functionality is claimed.

The proposed application may also introduce multilingual question support, including German and Japanese, to improve accessibility for learners studying foreign languages.

AI-generated explanations are another possible enhancement. The system may use AI to generate additional explanations and learning suggestions based on the selected question. However, generated content should undergo correctness review and be associated with the relevant question version. It should not automatically replace the established answer key without verification.

11.2 Possible Integrations and Optimizations

The proposed system will introduce authenticated cloud synchronization to maintain assessment records across multiple devices. The synchronization mechanism will include retry handling, conflict resolution, and account isolation to ensure that each learner's information remains associated with the correct account.

A stable attempt identifier will be introduced to distinguish individual assessment sessions. This identifier will help prevent duplicate records when synchronization requests are repeated because of network failures or application restarts.

The leaderboard functionality will be improved to support meaningful comparisons between learners. Rankings will be generated using comparable assessment conditions, such as subject, question count, and assessment type. This will reduce the limitations associated with combining unrelated attempts into a single ranking.

Database indexing will also be considered to improve data retrieval efficiency. Some possible index definitions include:

Proposed Index	Purpose
Question(topic_id, difficulty)	Supports filtering questions according to topic and difficulty.
QuizAttempt(learner_id, completed_at)	Supports retrieving learner assessment history in chronological order.
Topic(subject_id)	Supports retrieving topics associated with a selected subject.

These indexes represent proposed database optimization choices and have not been evaluated through performance benchmarks. Their effectiveness will depend on the actual database structure, query patterns, and volume of assessment records.

Representative datasets and database query plans should be used to evaluate the usefulness of each index. The evaluation should also consider additional storage requirements and the effect of indexes on insert and update operations.

Recording functionality will be reviewed according to the actual requirements of the assessment process. If recording is retained, the system will include explicit learner consent, defined retention periods, access restrictions, secure handling, and reliable release of camera and microphone resources.

Accessibility improvements will also be introduced to make the application easier to use for different learners. These improvements will include clearly labelled input fields, keyboard navigation, visible focus indicators, readable content, and feedback mechanisms that do not depend exclusively on colour.

The proposed enhancements will gradually transform the current browser-based quiz prototype into a more structured, persistent, secure, and adaptive learning platform. Each enhancement will be implemented and tested independently before integration into the complete application.

CHAPTER 12 CONCLUSION

12.1 Summary of the Project

The Exam Preparation App with Adaptive Quizzes is designed to provide an interactive learning environment that combines subject-based examination practice with database engineering concepts. The project focuses on helping learners practise questions, evaluate their knowledge, understand incorrect answers, and monitor their assessment performance.

The supplied browser-based prototype demonstrates the basic examination workflow, including subject selection, question selection, immediate answer feedback, explanations, grading, local result storage, learning resources, and mandatory recording functionality.

The proposed extended system introduces a broader Android-based architecture using Kotlin, Room Database, Firebase, and Item Response Theory (IRT). These technologies are intended to support persistent assessment records, centralized data management, personalized question selection, and synchronization across multiple devices.

The project report distinguishes the functionalities implemented in the available prototype from the features proposed for future development. It also presents the database schema, system architecture, representative SQL queries, data integrity requirements, testing procedures, deployment instructions, and future enhancements.

The proposed database design provides a structured foundation for maintaining learner information, subject details, question banks, quiz attempts, and individual responses. It also establishes a development pathway for introducing reliable data persistence, transaction management, and controlled synchronization.

Overall, the project demonstrates the application of database engineering principles in an educational assessment environment while providing a foundation for further development of an adaptive learning platform.

12.2 What Was Achieved

The project successfully established a browser-based examination practice workflow using the supplied HTML implementation. The available application supports learner entry, subject-based question selection, quiz execution, answer evaluation, immediate feedback, result generation, and local storage of assessment summaries.

The project also includes learning resources through embedded educational links and a mandatory recording mechanism using camera and microphone permissions. These functionalities were examined through limited browser-based functional verification using synthetic learner information.

The documentation provides a detailed explanation of the application's architecture, existing functionalities, challenges, limitations, and proposed improvements. It also includes a normalized relational database proposal, representative SQL statements, integrity constraints, testing scenarios, deployment procedures, and future acceptance criteria.

The testing section records the observed results of the available functional checks without treating proposed features as completed implementations.

The available evidence does not establish that the complete adaptive Android application or distributed database backend has been implemented. Further development requires Android source code, persistent response records, calibrated question parameters, authenticated cloud services, and integration testing.

Thus, the achieved work provides both a functional browser-based demonstration and a structured technical foundation for the proposed extended system.

12.3 Learning Outcomes

The project provided practical understanding of the relationship between frontend application functionality and backend data management. It demonstrated how learner interactions, question selection, answer evaluation, and result generation can be connected within an examination workflow.

Through the proposed database design, the project illustrates the importance of separating question information from learner attempts and individual responses. This separation supports structured data storage, reduces unnecessary duplication, and enables meaningful retrieval of assessment information.

The project also demonstrates how SQL operations such as joins, filtering, grouping, and aggregation can be applied to generate learner performance reports and topic-level analysis.

Another important learning outcome is understanding the difference between browser-local storage and relational database persistence. Browser storage is suitable for simple local demonstrations, whereas a database-based architecture provides stronger support for structured records, relationships, and transaction management.

The proposed Android and cloud architecture further introduces the concepts of Room database management, Firebase synchronization, database migration, authentication, authorization, and distributed data consistency.

The adaptive learning proposal provides an introduction to Item Response Theory and the importance of question calibration, learner ability estimation, and difficulty-based question selection.

The project also emphasizes the importance of software testing, security, data validation, accessibility, and technical documentation. The next development phase should demonstrate each newly implemented component through appropriate evidence, including a working Room database schema, migration test results, an adaptive question selection trace, and successful synchronization records.

In conclusion, the project connects database engineering concepts with practical educational application development and establishes a foundation for building a more reliable, personalized, and scalable examination preparation system.

CHAPTER 13 REFERENCES

Project sources

[1] Project Team, Database Project Review (2) (4).pptx, First Review Presentation, 16 August 2026, 12 slides. Source for project objectives, team roles, guide information, and proposed system architecture.

[2] Project Team, Database Abstract (2) (1).pdf, Project Abstract Submission Form, Course Code: 25CS1302E, Academic Year 2026–2027, Trimester 4, 2 pages. Source for project scope, team information, and proposed technology stack.

[3] Project Team, Exam Preparation App Final (3).html, Supplied Browser-Based Implementation. Source for implemented application features, JavaScript functions, quiz evaluation, scoring, and browser-local storage.

Technical Documentation

[4] Android Developers, “Save Data in a Local Database Using Room.” Available at: <https://developer.android.com/training/data-storage/room>

[5] Firebase Documentation, “Access Data Offline.” Available at: <https://firebase.google.com/docs/firestore/manage-data/enable-offline>

[6] SQLite, “SQLite Is Transactional.” Available at: <https://www.sqlite.org/transactional.html>

[7] MDN Web Docs, “MediaRecorder API.” Available at: <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>

Technical documentation accessed on 2 October 2026. References [4]–[7] provide technical information supporting the proposed architecture and design. They do not establish that these technologies have been fully implemented in the current project.

Formatting Reference

[8] SAMPLE Project Documentation (1) (1).pdf, Supplied Academic Project Report, 109 pages. Used as a reference for report structure, front matter, chapter organization, index formatting, typography, and appendix categories. Unrelated project information and claims were excluded from the present documentation

CHAPTER 14 APPENDICES

Appendix A Application Screenshots

The following images are captured from the supplied HTML with synthetic learner data. They illustrate the browser prototype, not the proposed Android implementation.

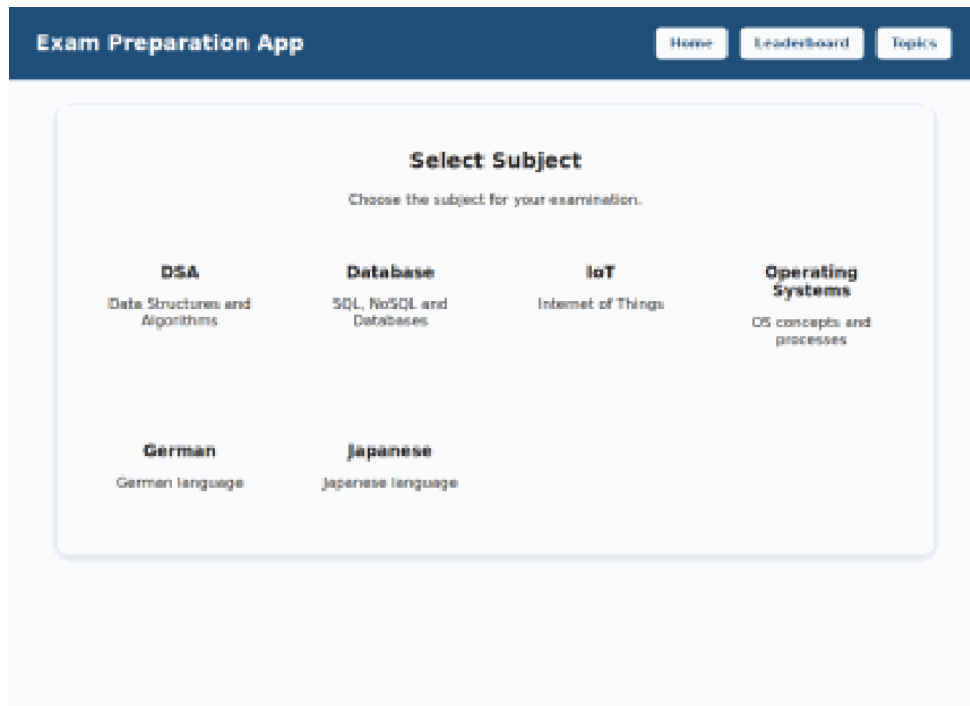


Figure A.1 Subject selection screen

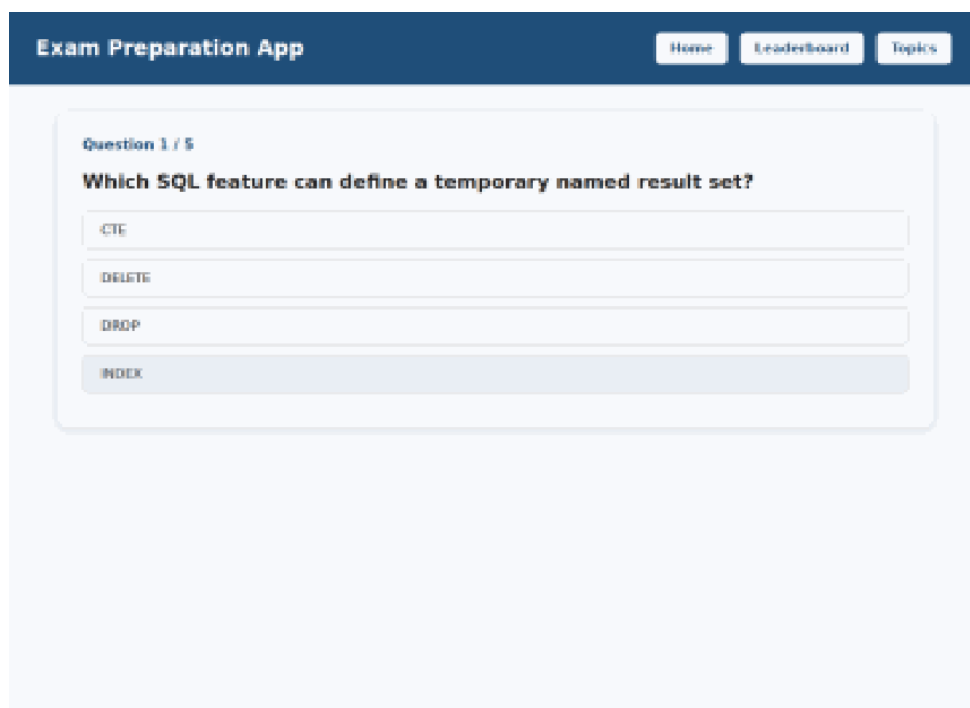


Figure A.2 Question screen during the demonstration attempt

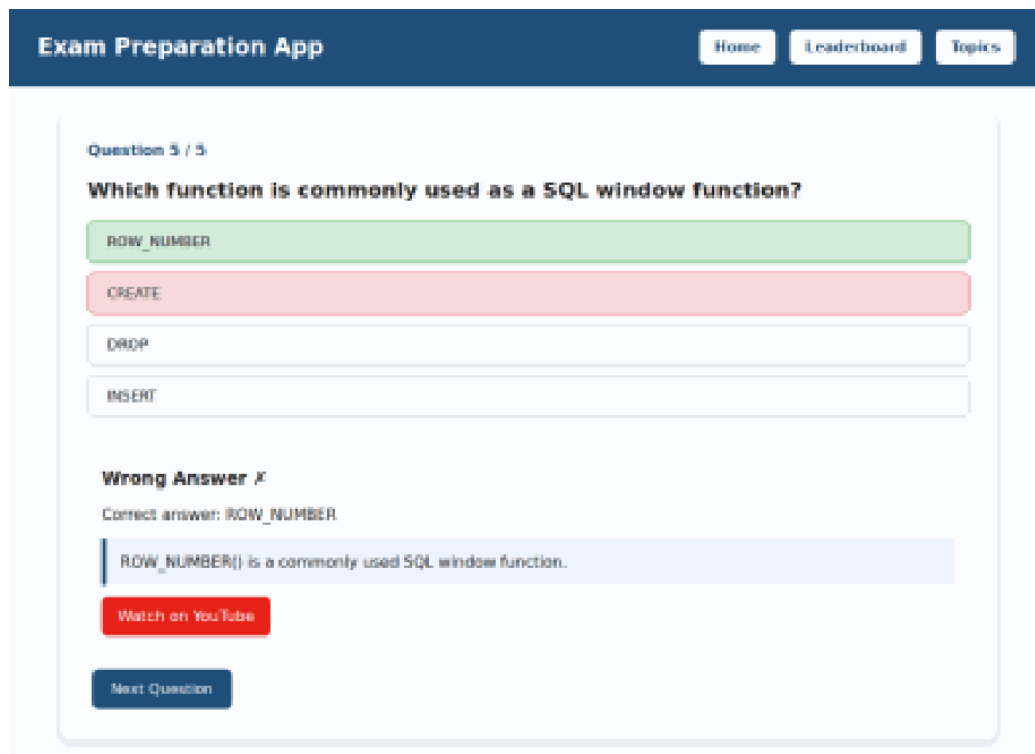


Figure A.3 Feedback and explanation after an incorrect answer

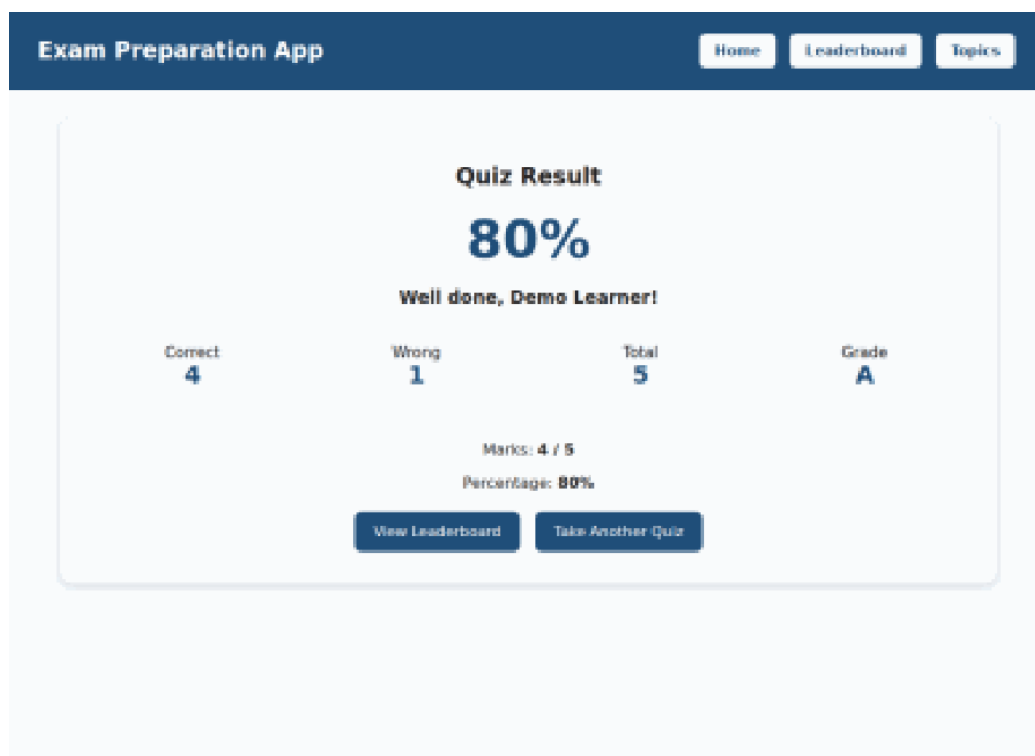


Figure A.4 Demonstration result with four correct answers out of five

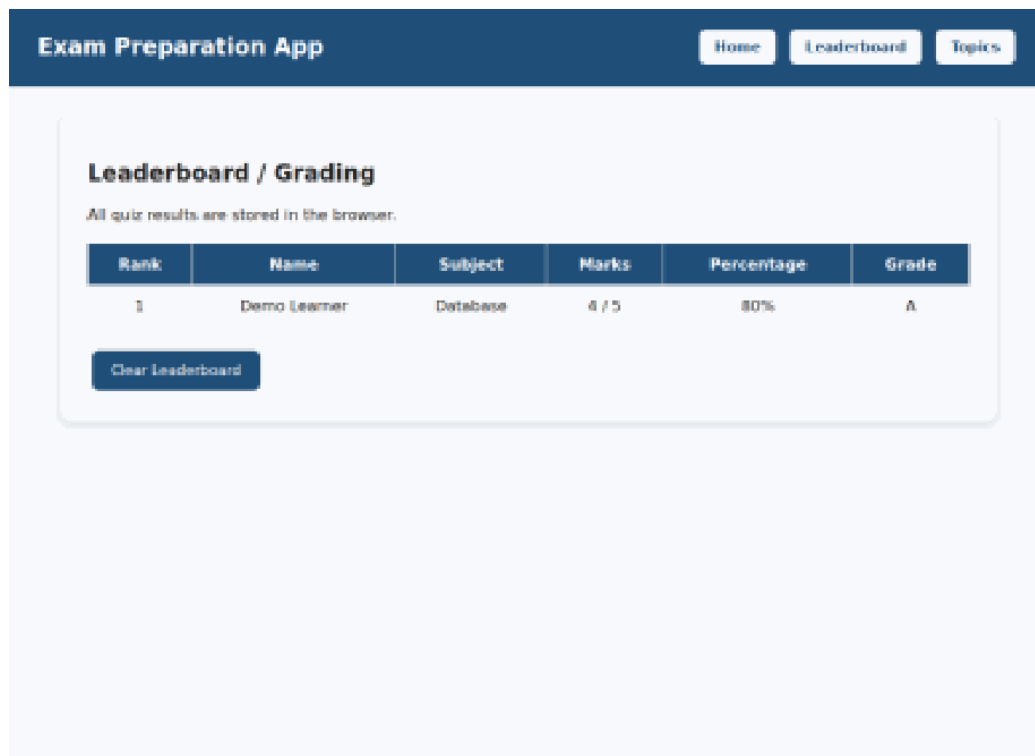


Figure A.5 Browser-local leaderboard with a demonstration result

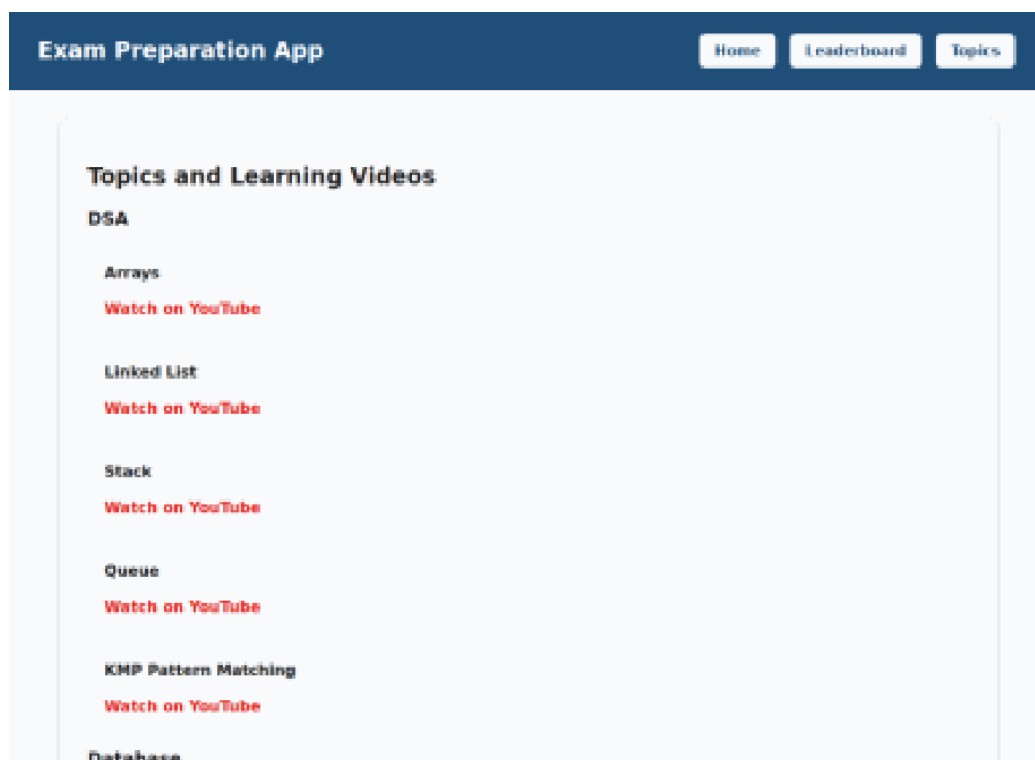
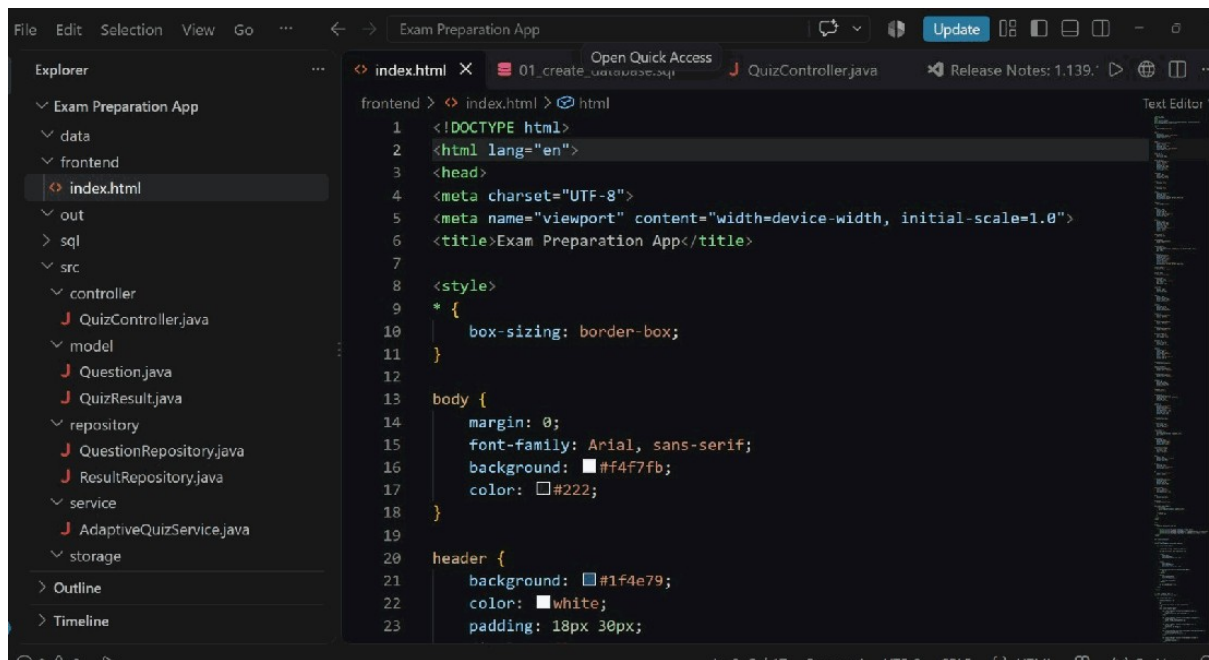


Figure A.6 Topics and learning links

Appendix B Sample Code Snippets



The screenshot shows a code editor with the following content:

```
File Edit Selection View Go ... Exam Preparation App
index.html X 01_create_database.sql QuizController.java Release Notes: 1.139.1
Explorer
  Exam Preparation App
    data
    frontend
      index.html
    out
    sql
    src
      controller
        QuizController.java
      model
        Question.java
        QuizResult.java
      repository
        QuestionRepository.java
        ResultRepository.java
      service
        AdaptiveQuizService.java
      storage
    Outline
    Timeline
  frontend > index.html > html
  1 <!DOCTYPE html>
  2 <html lang="en">
  3 <head>
  4 <meta charset="UTF-8">
  5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
  6 <title>Exam Preparation App</title>
  7
  8 <style>
  9 * {
 10   box-sizing: border-box;
 11 }
 12
 13 body {
 14   margin: 0;
 15   font-family: Arial, sans-serif;
 16   background: #f4f7fb;
 17   color: #222;
 18 }
 19
 20 header {
 21   background: #1f4e79;
 22   color: white;
 23   padding: 18px 30px;
```

```
index.html 01_create_database.sql X QuizController.java Release Notes: 1.139.0
sql > 01_create_database.sql
1 CREATE DATABASE exam_preparation_app;
2
3 USE exam_preparation_app;
4
5 CREATE TABLE students (
6     student_id INT PRIMARY KEY AUTO_INCREMENT,
7     student_name VARCHAR(100) NOT NULL,
8     email VARCHAR(150) UNIQUE
9 );
10
11 CREATE TABLE subjects (
12     subject_id INT PRIMARY KEY AUTO_INCREMENT,
13     subject_name VARCHAR(100) NOT NULL UNIQUE
14 );
15
16 CREATE TABLE questions (
17     question_id INT PRIMARY KEY AUTO_INCREMENT,
18     subject_id INT NOT NULL,
19     question_text VARCHAR(500) NOT NULL,
20     difficulty VARCHAR(20) NOT NULL,
21     correct_answer VARCHAR(255) NOT NULL,
22     FOREIGN KEY (subject_id)
23         REFERENCES subjects(subject_id)

```

Ln 20, Col 37 Spaces: 4 UTF-8 CRLF

```
File Edit Selection View Go ... Exam Preparation App
index.html 01_create_database.sql X QuizController.java Release Notes: 1.139.0
frontend > index.html > html > head > style > header
2 <html lang="en">
3 <head>
4 <style>
20 header {
24     display: flex;
25     justify-content: space-between;
26     align-items: center;
27 }
28
29 header h1 {
30     margin: 0;
31     font-size: 24px;
32 }
33
34 header button {
35     background: white;
36     color: #1f4e79;
37     border: none;
38     padding: 9px 16px;
39     border-radius: 6px;
40     cursor: pointer;
41     font-weight: bold;
42 }

```

Appendix C Installation and User Guide

Installation and Setup

1. Retain the original HTML and create a working copy named index.html in a local folder.
2. Start a static server from that folder, for example `python -m http.server 8000`. Open `http://localhost:8000` in a compatible browser.
3. Confirm that a camera and microphone are available. Use only demonstration data during classroom reviews unless real data collection has been explicitly agreed.
4. Check the browser console for errors. If media permission was denied, change the site permission and retry. Internet access is required for YouTube resources.

User Manual

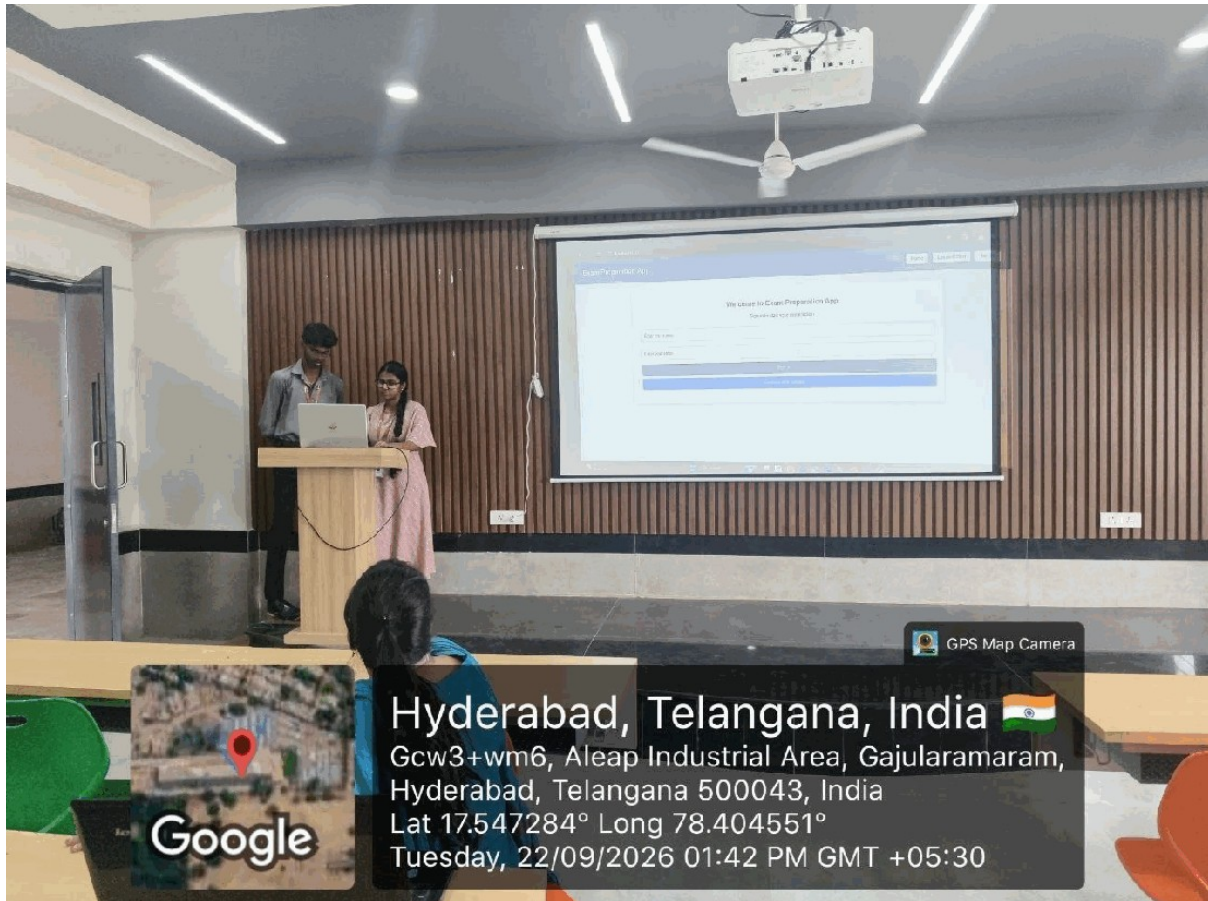
1. Enter your name and email, then select Sign In. The Google-labelled alternative is only a simulation in this version.
2. Select DSA, Database, IoT, Operating Systems, German, or Japanese. Choose the requested number of questions; the available question bank may reduce the actual count.
3. Select Start Camera & Recording. Once the app reports active recording, select Start Quiz.
4. Choose one option for each question. Read the explanation and use Next Question to continue.
5. Review your final score and grade. Open the leaderboard to view attempts saved in this browser. Use Topics for additional learning links.
6. Clear Leaderboard only when you intend to remove every result saved under the app's leaderboard key. Browser data does not provide a multi-device backup.

Troubleshooting

If results disappear, check whether the browser profile, site origin, or storage was changed. If fewer questions appear than requested, check the available bank size. A blank or old learning link can result from the feedback-link reset limitation described in Chapter 10.

Appendix D Geo Tagged Photographs with Guide





Photograph caption: _____

Date and location: _____

Guide and team members: _____

Appendix E Review Form and Guide Signatures

Project: Exam Preparation App with Adaptive Quizzes

Section: 7 Team: 3

Guide: Dr. Spandana. S

Course: 25CS1302E

Review field	To be completed
Review date	
Progress demonstrated	
Database design feedback	
Implementation feedback	
Testing observations	
Corrections required	
Next review date	

Guide remarks:

Guide signature: _____ Date: _____

Team signatures:

Konda Vaishali - 2520030117 _____

P. Bindhu Madhavi - 2520030077 _____

Alaparthi Nava Harsha - 2520030493 _____